HERMES: High-Performance Private Decoding for Large Language Models

MohammadHossein Olyaiy, Cora Shao, Dingqing Yang,
Mohammad Shahrad, and Mieszko Lis
The University of British Columbia
{mohamadol, corashao, dingqingy, mshahrad, mieszko}@ece.ubc.ca

Abstract

Privacy-preserving inference (PI) enables users to query large language models (LLMs) without revealing their prompts, but incurs substantial computation and communication overhead. Prior PI systems primarily optimize prefill and time to first token (TTFT). This suits applications that need only a single output token, yet most real-world use cases require multi-token responses. Such autoregressive workloads are dominated by time per output token (TPOT) instead.

For the first time, we investigate generating full multi-token answers for real prompts under PI. We show that PI decode is hindered by two bottlenecks: (i) nonlinear kernels that require many interactive communication rounds, and (ii) attention protocols that underutilize ciphertext capacity and repeatedly re-encrypt or transmit the growing key and value (KV) cache.

We present HERMES, a PI system optimized for LLM decoding. HERMES replaces high-round nonlinear protocols with compute- and bandwidth-intensive alternatives that require fewer communication rounds. It also introduces a novel matrix-multiplication protocol based on homomorphic encryption that keeps the KV cache encrypted on the server and uses grouped-query attention (GQA) to improve ciphertext utilization. On Llama 3.1 8B Instruct, across the network conditions and prompts used in our evaluation, HERMES lowers TPOT by up to 2.1× and total PI latency by up to 1.9×.

1 Introduction

Large language models (LLMs) [5, 14, 28, 63] are popular cloud-hosted services. Yet, in sensitive domains like healthcare and finance, handing user data to a remote server raises privacy issues [4]. Privacy-preserving inference (PI) protects users' prompts and results from the inference service; however, the cryptographic primitives it relies on, such as Homomorphic Encryption (HE) [11], Oblivious Transfer (OT) [55], and Function Secret Sharing (FSS) [10], add significant processing and communication overheads [33, 34, 54]. Several works have driven these overheads down [2, 18, 19, 24, 34, 39, 41, 44, 54], putting LLM PI seemingly within practical reach.

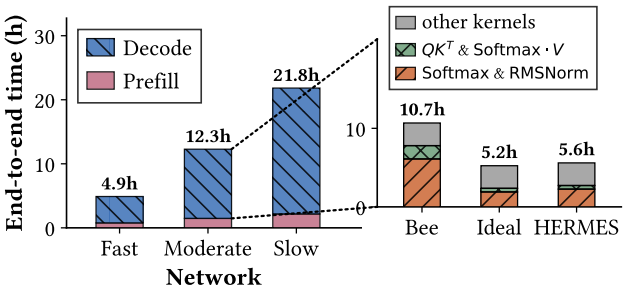


Figure 1: Solving a GSM8K math prompt with Llama 3.1 8B Instruct under various networks (Section 7.2), using a BumbleBee-like PI system extended to support autoregressive decoding. *Ideal* assumes a single communication round for Softmax and RMSNorm and the minimum number of ciphertexts for attention ($\mathbf{Q} \cdot \mathbf{K}^\top$ and Softmax $\cdot \mathbf{V}$) during decode.

Despite this progress, no PI system to date has been shown to do what many use cases actually require from an LLM: take a real prompt and generate a complete answer. State-of-the-art frameworks have mostly focused on producing a single output token. Their metric is the *time to first token* (TTFT), often implicitly: they either consider encoder-only models [34, 52, 54] such as BERT [22] and ViT [26], or consider generative LLMs [33, 39, 41, 67, 69] but only investigate generating the first token. Their optimizations therefore target the *prefill* phase, leaving *autoregressive decode*, the phase that generates every subsequent token and dominates the runtime of most queries (see Figure 1), unexamined.

In this work, we instead investigate running PI as a full generative service, where the client sends an encrypted prompt, inference runs as a two-party protocol between the client and the server, and the client receives a complete (encrypted) response one token at a time.

Problems. We extend state-of-the-art PI frameworks to support decoding (for instance, by adding a straightforward secret-shared key and value (KV) cache) and characterize their performance. Interestingly, we find several inefficiencies that

are negligible, or do not even arise, in prefill, yet dominate the *time per output token* (TPOT) during decode, where each token depends on the previous ones and must therefore be generated one at a time.[1] Plaintext serving treats TPOT as a first-class metric alongside TTFT [72]; in PI, it has received little attention.

Specifically, nonlinear kernels such as Softmax and RMSNorm require many sequential client–server communication rounds. In prefill, this round latency is amortized over the long prompt and hidden behind larger bottlenecks such as HE-based matrix multiplications (Matmuls) [34, 39, 54]; in decode, it is paid per token, and with the prefill bottlenecks gone, it quickly dwarfs every other cost.

The linear kernels of attention are similarly inefficient: executed naively, they waste significant bandwidth during decode. First, a single new token per step leaves HE ciphertexts severely underutilized, whereas prefill fills them with an entire prompt's worth of tokens. Second, efficiently managing the KV cache under PI is a challenge that has received little attention in prior work [39, 41, 44, 54]. The trivial solution, secret-sharing the cache between the parties, in many common settings requires re-transmitting the entire cache at each step. Some works instead use more flexible HE encodings (e.g., based on the Chinese remainder theorem, CRT) [69], which improve ciphertext utilization and simplify KV cache management. However, such techniques are notorious for slowing down HE computation (see [37, 39] and our evaluation in Section 8.4), shifting the problem from slow communication to slow computation.

Together, these kernels dominate decode: in Figure 1 (right), Softmax, RMSNorm, $\mathbf{Q} \cdot \mathbf{K}^T$, and Softmax $\cdot \mathbf{V}$ account for 7.8 hours, more than 70% of the total decode time. As the *Ideal* bar shows, optimizing away their rounds and communication would make decode about 2× faster.

Contributions. We address these bottlenecks with protocols designed for decode's regime: small data volumes, latency-bound communication, and a KV cache that grows by one row per step. First, we accelerate the nonlinear kernels. Their client–server communication rounds are inherently sequential, so their latency cannot be hidden with pipelining techniques. Batching multiple requests can help but is not feasible in many scenarios, as it significantly increases the user-facing latency (e.g., days instead of hours, see Section 8.5).

We therefore take a different approach and systematically *trade rounds for additional compute or bandwidth*. Specifically, we (i) precompute normalization constants that empirically remain within a bounded range, (ii) employ techniques such as FSS [10], which uses fewer rounds at the cost of higher compute and bandwidth, and (iii) increase HE noise budgets to limit refresh rounds at the price of more compute-intensive HE operations (see Section 5 for details). Our evaluation shows these trade-offs reduce the nonlinear portion of TPOT by up to 2.2× (Section 8.2).

Next, we accelerate attention by reducing its communication volume. We propose *a novel HE-Matmul protocol* that (i) packs multiple grouped-query attention (GQA) queries sharing the same KVs into a single ciphertext vector, increasing ciphertext utilization, and (ii) lets the KV cache remain encrypted and grow on the server: at each decode step, the client sends only one delta ciphertext to update the cache, instead of recomputing [18, 34, 41, 54] or re-transmitting [39] the entire cache even though only one row changes per token. Moreover, our protocol builds on the faster prior HE-Matmuls that use the direct polynomial encoding [34, 39] instead of CRT, so it reduces communication without inflating HE computation.

We build HERMES, a proof-of-concept implementation available at https://github.com/compstruct/hermes, and evaluate it on Llama 3.1 8B Instruct [32, 63] with real-world math reasoning prompts from the GSM8K dataset [21]. To the best of our knowledge, HERMES is the first PI framework to run an LLM on real prompts, generating every token to the full completion. Compared to the state-of-the-art frameworks that we extend to support decoding, HERMES reduces end-to-end latency by 1.4–1.9×, depending on the prompt and network.

2 Background

2.1 Notation

We adopt the following conventions throughout the paper; the recurring symbols are summarized in Table 1.

Sets, rings, and fields. Standard algebraic structures are written in blackboard bold. $\mathbb{Z}$ denotes the integers, $\mathbb{R}$ the reals, and $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ the integers modulo q. Our protocols operate over two such structures. The native ring of the baseline framework [39] we build on is the *power-of-two ring* $\mathbb{Z}_{2^k}$, which is well-suited for OT protocols. When needed, we also use the *prime field* $\mathbb{Z}_p$. HE ciphertexts live in the *negacyclic polynomial ring* $\mathcal{R}_{q,n} = \mathbb{Z}_q[X]/(X^n+1)$, where n is a power-of-two polynomial degree and q is the ciphertext modulus. When n is clear from context we abbreviate this as $\mathcal{R}_q$. Finally, X is reserved for the polynomial indeterminate.

Scalars, vectors, matrices, and tensors. Italic letters denote scalars, whether lowercase (e.g., d) or uppercase (e.g., N, T). Vectors are marked with an arrow (e.g., $\vec{v}$). Bold uppercase letters denote matrices and higher-order tensors (e.g., $\mathbf{Q}$). We indicate dimensions with set-membership notation: $\mathbf{Q} \in \mathbb{R}^{h_q \times N \times d}$ is a three-axis tensor with h_q heads, N rows, and d columns per head. The entry of $\mathbf{Q}$ at head i, row r, and column c is $\mathbf{Q}_{i,r,c}$.

[1] Speculative decoding [43] can partially mitigate this by verifying multiple tokens in parallel, but typically only a few tokens (e.g., 3) in advance.

Table 1: Notation used in this manuscript.

Symbol	Description
LLM	
N	Number of input tokens in the current step
N_{prefill}	Number of prompt tokens (prefill input)
N_{kv}	KV cache length
T	Number of autoregressive decode tokens generated
d_h	Model hidden dimension
d	Per-head key/value dimension (d_h/h_q)
h	Number of key/value (KV) heads
h_q	Number of query heads (=h for MHA)
G	GQA group size (h_q/h query heads per KV head)
Crypto	
n	RLWE polynomial ring degree
p and q	Plaintext and ciphertext moduli
RLWE(m)	RLWE plaintext polynomial of message m, in $\mathcal{R}_p$
$[\![\text{RLWE}(m)]\!]$	RLWE ciphertext of m, in $\mathcal{R}_q^2$
k	Bit-width of the secret-sharing ring $\mathbb{Z}_{2^k}$
S_{ct}	Size of a single ciphertext (bytes)
System	
t_{proc}	Time to process one HE operation
BW	Network bandwidth
t_{comm}	Time to transmit one HE ciphertext (S_{ct}/BW)
t_{rtt}	Network round-trip time

2.2 Cryptographic Primitives

We provide a brief introduction to the cryptographic tools we use and the trade-offs they entail. A more detailed background is provided in Appendix E.

Additive secret sharing hides an intermediate result x by splitting it between the two parties as random shares $\langle x\rangle_0$ and $\langle x\rangle_1$, with $\langle x\rangle_0 + \langle x\rangle_1 \equiv x \pmod{2^k}$. We use **OT** [38, 55] and **FSS** [10] to realize nonlinear operations such as comparisons and multiplexers. OT and FSS sit at opposite ends of a rounds-versus-bandwidth trade-off: OT generates its correlated randomness with little bandwidth and few rounds but can take many interactive rounds during evaluation, whereas FSS requires pre-generating large keys yet completes each evaluation in a single round.

Finally, we use **HE** [11, 27] for operations that decompose into additions and multiplications (e.g., polynomial evaluation). Like most prior work [34, 37, 39, 48, 54], we use HE schemes built on Ring-Learning-with-Errors (RLWE) [12, 20, 46, 50]. A message m is first encoded into a plaintext polynomial RLWE(m) $\in \mathcal{R}_p$, which can later be encrypted into a ciphertext $[\![\text{RLWE}(m)]\!] \in \mathcal{R}_q^2$. Evaluating on ciphertexts increases the ciphertext noise, and decryption succeeds only while this noise stays under a budget controlled by q/p. HE parameters (i.e., n, q, and p) set a *depth-versus-rounds* trade-off: lighter parameters are fast per operation but force frequent interaction to refresh the noise; heavier ones support evaluating deeper circuits but have a higher per-operation latency.

2.3 HE-Matmul

We adopt and extend the HE-Matmul protocol from Bumble-Bee [39]. We briefly introduce the protocol and discuss its trade-offs. For more details see Appendix F or [39].

The protocol exploits the fact that multiplication in $\mathcal{R}_{q,n}$ is a convolution of coefficient vectors: products of coefficients accumulate wherever their indices sum to the same value, with a sign flip when that sum wraps past n (since $X^n \equiv -1$). The input matrices are first tiled into sub-blocks $\mathbf{A} \in \mathbb{R}^{d_0 \times d_1}$ and $\mathbf{B} \in \mathbb{R}^{d_1 \times d_2}$ with $d_0 \cdot d_1 \cdot d_2 \le n$. Then for each pair of tiles, the encryptor encrypts $\mathbf{A}$ into $[\![\text{RLWE}(\mathbf{A})]\!]$ and sends it to the evaluator; the evaluator encodes $\mathbf{B}$ as a plaintext polynomial RLWE($\mathbf{B}$) and homomorphically computes $[\![\text{RLWE}(\mathbf{C})]\!] = [\![\text{RLWE}(\mathbf{A})]\!] \boxtimes \text{RLWE}(\mathbf{B})$.

Unlike CRT encoding techniques, which provide more compact ciphertexts but require slow permutations under HE [37, 39], this protocol encodes each matrix tile directly into the polynomial coefficients, enabling faster HE operations. Concretely, writing $i \in [0, d_0)$, $j \in [0, d_1)$, and $k \in [0, d_2)$ for the indices, the entries are placed at

$$\text{pos}_{\mathbf{A}}(i,j) = i \cdot d_1 d_2 - j + n \cdot \mathbf{1}[i{=}0 \wedge j{>}0], \tag{1}$$

$$\text{pos}_{\mathbf{B}}(j,k) = k \cdot d_1 + j, \tag{2}$$

with $\mathbf{A}_{0,j}$ negated for $j > 0$ so that the wrap-around's sign flip cancels out. These positions satisfy $\text{pos}_{\mathbf{A}}(i,j) + \text{pos}_{\mathbf{B}}(j,k) = i \cdot d_1 d_2 + k \cdot d_1 \pmod{n}$ *independently of* j, so the convolution accumulates the d_1 products $\mathbf{A}_{i,j}\mathbf{B}_{j,k}$ into the single coefficient $\text{RLWE}(\mathbf{C})[\,i \cdot d_1 d_2 + k \cdot d_1\,] = \mathbf{C}_{i,k}$ as needed by the matrix product $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$.

2.4 LLMs and Decoding

Modern LLMs [14, 62, 63] are based on transformers [65]. Generative LLMs, the focus of our work, stack identical decoder blocks, each consisting of multi-head attention and a feed-forward network (FFN) with normalizations in between. We briefly describe the operations relevant to our work; see [63, 65] for a full treatment.

Multi-head attention. Given an input $\mathbf{X} \in \mathbb{R}^{N \times d_h}$, learned projections produce queries, keys, and values, reshaped into per-head tensors $\mathbf{Q} \in \mathbb{R}^{h_q \times N \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{h \times N \times d}$, where h_q is the number of query heads, h the number of key/value heads, and $d = d_h/h_q$ the per-head dimension. Modern LLMs use *grouped-query attention* (GQA) [1], where $h < h_q$ and each KV head is shared across h_q/h query heads, reducing the size of the KV state. For each query head i with corresponding KV head $j(i)$, the attention output is

$$\mathbf{A}_i = \text{Softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_{j(i)}^{\top}}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V}_{j(i)}, \tag{3}$$

where $\mathbf{M}$ is a causal mask and Softmax is applied row-wise. The per-head outputs are concatenated and passed through an output projection.

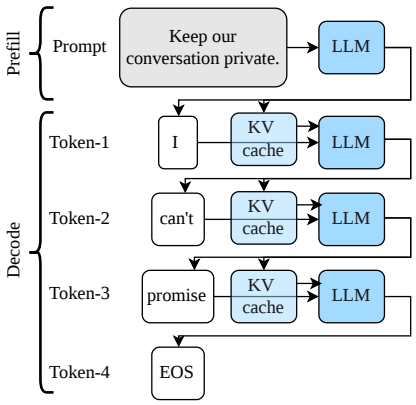


Figure 2: Generative LLM inference where a *prefill* pass processes the prompt and produces the first token. Autoregressive *decode* then samples one token at a time, reusing a growing KV cache.

Normalization. Each sub-layer is preceded by a normalization step. The models we target use RMSNorm [62, 63]:

$$\mathsf{RMSNorm}(\vec{x}) = \frac{\vec{x}}{\sqrt{\mathrm{mean}(\vec{x}^2) + \varepsilon}} \cdot \vec{w},$$

where $\vec{w}$ is a learned per-channel scale and ε a small constant for numerical stability.

Autoregressive decoding. LLMs generate text in two phases as shown in Figure 2. The *prefill* phase processes the user's prompt of length N_{prefill} (i.e., $\mathbf{X} \in \mathbb{R}^{N_{\text{prefill}} \times d_h}$) in a single forward pass, producing the first output token. The *decode* phase then produces tokens one at a time: at each step the previously sampled token is fed back as input ($\mathbf{X} \in \mathbb{R}^{1 \times d_h}$), and the matrix multiplications of attention and the FFN reduce to matrix–vector products. This shape change is the main reason decode warrants a separate set of optimizations (Section 4).

KV cache. Since $\mathbf{K}$ and $\mathbf{V}$ rows from earlier tokens are reused at every subsequent decode step, standard practice is to avoid regenerating them by caching them in a *key–value (KV) cache* [60]. At each step the model computes only the new rows $\mathbf{K}_{\text{new}}, \mathbf{V}_{\text{new}} \in \mathbb{R}^{h \times 1 \times d}$, appends them to the cache, and reuses the unchanged earlier rows. We denote the KV cache length at each step as $N_{\text{kv}} = N_{\text{prefill}} + T$, where T is the number of tokens already generated.

3 Privacy and Threat Model

3.1 Threat Model and Privacy Goals

We consider the standard two-party PI setting [34, 37, 40, 54], in which a *client* owns a private prompt and a *server* owns proprietary model weights. At the end of the protocol, the client learns the generated tokens and the server learns nothing beyond the public information below. As in prior work [34, 37, 39, 41, 54], public information includes the lengths of the prompt and generated output, and the model structure (e.g., the number of layers and their dimensions, but not the learned weights).

We inherit the threat model of prior work [34, 37, 39, 40, 54]: a *static, semi-honest* adversary corrupting one of the two parties. The primitives we reuse from prior work—HE-Matmul [39]; OT-based protocols [68] for truncation [57], comparison [37, 57], and dealer-free FSS key generation [23, 66]; and FSS protocols for rsqrt and Max [33]—are all proven secure in this threat model. Moreover, they consume and produce additive secret shares, so intermediate results remain hidden across kernel boundaries as well. Finally, the constants we precompute are derived from the server's calibration data, never from a client's query, and never leave the server; they thus reveal nothing about the prompt or the model.

3.2 Setting

HERMES operates in the standard interactive two-party setting [34, 37, 40, 48, 54]: the client and server collaborate to privately execute nonlinear operations such as comparison, and to refresh the noise of HE ciphertexts, avoiding expensive bootstrapping. Works that instead run inference entirely on the server [69, 70] must approximate the nonlinear operations, and it is unclear whether those approximations preserve accuracy over the long generation chains of autoregressive decode, where each token depends on the previous ones.

Beyond interactivity, we make three further choices that favor realistic deployment over easy speedups. First, we report end-to-end latency rather than splitting off an offline phase, since preprocessing can rarely be fully hidden in practice [30]. Second, we assume no trusted dealer—one would accelerate FSS but at the cost of a weaker threat model. Third, we fix the client as the encryptor (RLWE key holder) and the server as the evaluator throughout HE protocols, matching the setting used in most prior work [40, 48, 54]; this is sensible because client devices may have hardware support for encryption and decryption [64] but rarely for HE evaluation [59]. BumbleBee [39] instead swaps these roles to lower communication; we obtain the savings through protocol design and, despite the stricter setting, still substantially outperform role-swapping designs (Section 8).

4 Motivation

In this section, we investigate why decode is inefficient in prior work. We profile BumbleBee [39], the state-of-the-art PI framework, to identify the kernels that dominate decode time, and then model them mathematically and analyze them

empirically to explain why they scale poorly relative to prefill. BumbleBee does not support autoregressive generation natively. We therefore extend it to be decode-aware: it tracks generation progress to distinguish prefill from decode steps, and after each layer it secret-shares the KV cache between the two parties, growing the cache incrementally as new rows are produced. All measurements use two virtual machines (VMs), acting as client and server, each with 64 vCPUs and 256 GB of memory. We use a link with 1 Gbps bandwidth (BW) and 10 ms round-trip time (RTT) as the *fast* network and one with 200 Mbps and 80 ms RTT as the *slow* network.

4.1 Profiling

Figure 3 shows the per-operation speedup ratio, $^{\text{TTFT}}/_{\text{TPOT}}$ (i.e., prefill latency over single-token decode latency), for Llama 3.1 8B with a 134-token prompt. Certain linear operations such as FFN achieve high speedups. However, there are two classes of kernels that see far less improvement, even on the fast network.

First, the nonlinear kernels that require many communication rounds see little speedup when going from $N_{\text{prefill}} = 134$ to $N = 1$ during decode (e.g., RMSNorm at 4.7×, Softmax at 2×), suggesting that the way nonlinear functions are evaluated privately is fundamentally unsuited to single-token decoding. Second, the linear attention kernels that involve the KV cache see limited speedup as well (e.g., $\mathbf{QK}^\top$ and $\text{Softmax} \cdot \mathbf{V}$ at 2.8×), indicating that the current HE-Matmul protocols for attention do not exploit the decode setting efficiently either.

Together, these two groups are the main bottleneck during decode (see Figure 1). Next, we mathematically model and profile these kernels to understand their behavior. Throughout the following analysis, we model kernel latency as the sum of compute, communication, and RTT terms:

$$T_{\text{kernel}} = T^{\text{compute}}_{\text{kernel}} + T^{\text{comm}}_{\text{kernel}} + T^{\text{rtt}}_{\text{kernel}} \tag{4}$$

4.2 Nonlinear Kernels

Nonlinear functions are typically implemented by combining OT-based protocols, such as a comparison tree to find the maximum of a vector, with HE-based polynomial approximations, such as approximating exponentiation [39, 54]. We focus on Softmax in this section, but the analysis generalizes to other nonlinear functions such as RMSNorm.

Softmax is computed via a hybrid HE-OT protocol over $h \cdot N$ rows, each of length N_{kv}. We model its latency following Equation (4), with: $T_{\text{row}}(N_{\text{kv}})$ denoting the compute time for one row, $C_{\text{row}}(N_{\text{kv}})$ the bytes transmitted per row, and $R(N_{\text{kv}})$ the total OT rounds required for one row.

$$T_{\text{softmax}} \approx \underbrace{hN \cdot T_{\text{row}}(N_{\text{kv}})}_{\text{compute}} + \underbrace{\frac{hN \cdot C_{\text{row}}(N_{\text{kv}})}{\text{BW}}}_{\text{comm}} + \underbrace{R(N_{\text{kv}}) \cdot t_{\text{rtt}}}_{\text{RTT}} \tag{5}$$

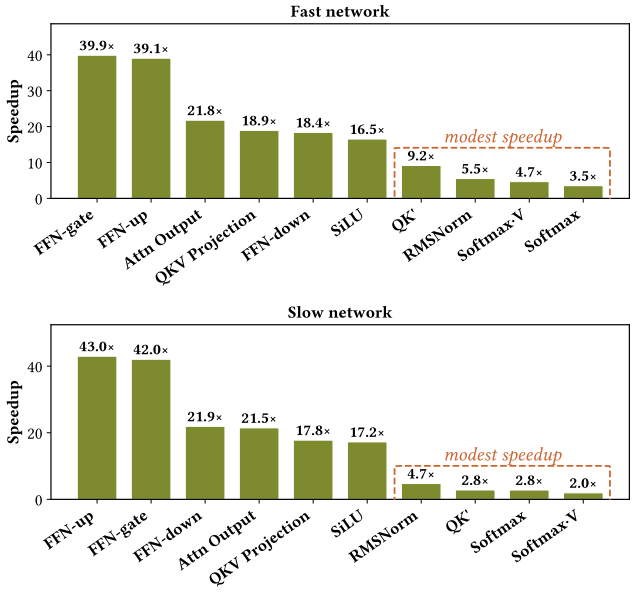


Figure 3: Per-operation speedup of one decode step over prefill (i.e., TTFT / TPOT) for Llama 3.1 8B with an $N_{\text{prefill}} = 134$ prompt, executed under the BumbleBee PI system [39].

Note that the number of rounds does not scale with h or N and therefore does not change when moving to decode (all rows can be batched within the same communication round in prefill). However, $R(N_{\text{kv}})$ itself is large; its cost is merely hidden in prefill behind the much larger compute and communication terms. In our baseline implementation [39, 47], Softmax requires ≈ 96 rounds per layer (varying slightly with N_{kv}) and RMSNorm requires ≈ 42 rounds per invocation (84 per layer); see Table 2 for details. This is problematic because during decode ($N = 1$), the compute and communication terms in Equation (5) shrink by a factor of N_{prefill}, while the RTT term remains unchanged. As a result, the RTT term dominates the latency of nonlinear kernels—especially on high-latency links.

Figure 4 confirms this empirically. We measure the per-layer Softmax latency during decode under fixed BW and varying RTT (1 ms to 60 ms). The relationship is strikingly linear: Softmax latency grows proportionally to RTT, and the slope increases with KV cache length (larger N_{kv} requires more rounds for the Max tree), while the other terms in Equation (5) contribute only a small constant offset (the y-intercept).

Goals. During decode, data sizes are small, so the compute and communication terms are negligible while RTT dominates. We therefore aim to design nonlinear protocols for Softmax and RMSNorm specifically tailored to the decode phase, prioritizing a low round count even at the cost of additional compute or bandwidth (Section 5).

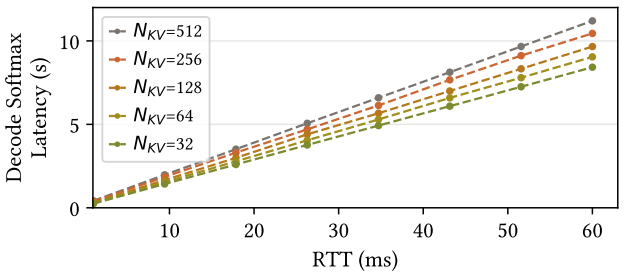


Figure 4: Llama 3.1 8B per-layer Softmax decode latency as a function of RTT for different KV cache lengths.

4.3 Linear Kernels of Attention

$\mathbf{QK}^\top$ and $\mathsf{Softmax} \cdot \mathbf{V}$ are both executed by the same HE-Matmul protocol; we therefore focus on $\mathbf{QK}^\top$. $\mathbf{QK}^\top$ is executed with a batched HE-Matmul where the batch size is the number of query heads (h_q), and the operand tensors are $\mathbf{Q} \in \mathbb{R}^{h_q \times N \times d}$ and $\mathbf{K}^\top \in \mathbb{R}^{h \times d \times N_{\text{kv}}}$. When several $\mathbf{Q}$ heads share the same $\mathbf{K}$ head due to GQA (i.e., $h < h_q$), $\mathbf{K}^\top$ is repeated h_q/h times. Because $\mathbf{Q}$ and $\mathbf{K}$ are secret-shared as $\mathbf{Q} = \langle \mathbf{Q} \rangle_0 + \langle \mathbf{Q} \rangle_1$ and $\mathbf{K} = \langle \mathbf{K} \rangle_0 + \langle \mathbf{K} \rangle_1$, expanding the product yields:

$$\mathbf{QK}^\top = \underbrace{\langle \mathbf{Q} \rangle_0 \langle \mathbf{K} \rangle_0^\top + \langle \mathbf{Q} \rangle_1 \langle \mathbf{K} \rangle_1^\top}_{\text{local}} + \underbrace{\langle \mathbf{Q} \rangle_0 \langle \mathbf{K} \rangle_1^\top + \langle \mathbf{Q} \rangle_1 \langle \mathbf{K} \rangle_0^\top}_{\text{requires HE}} \tag{6}$$

The two cross terms cannot be computed locally and require HE-Matmul calls. Therefore, the encryptor encrypts its shares ($\langle \mathbf{Q} \rangle_0$ and $\langle \mathbf{K} \rangle_0$) and sends them to the evaluator to compute the cross terms under HE. Following Equation (4), we model the HE-Matmul latency with compute, communication, and RTT terms:

$$\begin{aligned} T_{\mathbf{QK}^\top} \approx &\left(\text{Enc}_{\mathbf{Q}} + \text{Enc}_{\mathbf{K}} + \text{Mul}_{\mathbf{QK}^\top} + \text{Dec}_{\mathbf{QK}^\top} \right) t_{\text{proc}} \\ &+ \left(\text{Enc}_{\mathbf{Q}} + \text{Enc}_{\mathbf{K}} + \text{Dec}_{\mathbf{QK}^\top} \right) t_{\text{comm}} + t_{\text{rtt}}, \end{aligned} \tag{7}$$

where t_{proc} denotes the processing time per HE operation and t_{comm} the transmission time per ciphertext.

Low ciphertext utilization. Each HE ciphertext holds n coefficients (e.g., $n = 2048$), but during decode the query has only d elements to pack per head. For typical model dimensions ($d = 128$, so $d \ll n$) and $N = 1$, the ciphertexts become highly underutilized: we have $\text{Enc}_{\mathbf{Q}} = h_q \lceil \frac{d}{n} \rceil = h_q$ ciphertexts, each using only 6% of its coefficients. This inflates both encryption and communication costs. Importantly, the low utilization is not due to a lack of work—there is enough across the query rows of all h_q heads—but rather due to the current protocol, which has no support for packing multiple heads into the same ciphertext.

Redundant communication. A secondary issue arises if the goal is to have fixed client–server roles and refrain from evaluating HE computations on the client's device, which is our target and the standard arrangement in many settings (see Section 3.2). With a secret-shared KV cache, the client's entire share of the cache must then be re-encrypted and re-transmitted to the server at every step. This means $\text{Enc}_{\mathbf{K}} = h_q \left\lceil \frac{N_{\text{kv}} \cdot d}{n} \right\rceil$, as opposed to $\text{Enc}_{\mathbf{Q}} = h_q$, and incurs more encryption and communication for the cross term $\langle \mathbf{Q} \rangle_1 \langle \mathbf{K} \rangle_0^\top$ in Equation (6), since the server needs the encryption of the client's $\langle \mathbf{K} \rangle_0^\top$. To make matters worse, this cost grows with the KV cache at every decode step.

Table 2: Communication rounds per kernel invocation in the state-of-the-art PI framework BumbleBee [39] and in HERMES, reported for Llama 3.1 8B with $N_{\text{kv}} = 128$.

	BumbleBee		HERMES	
Component	**Softmax**	**RMSNorm**	**Softmax**	**RMSNorm**
HE-based				
polynomials	20	10	2	—
division / rsqrt	20	4	3	—
ring–field conversion	—	—	3	—
others	2	6	1	3
Subtotal	*42*	*20*	*9*	*3*
OT- / FSS-based				
Max	42	—	18	—
division / rsqrt	12	22	—	2
others	—	—	—	4
Subtotal	*54*	*22*	*18*	*6*
Total	**96**	**42**	**27**	**9**

Goals. We aim to design an HE-Matmul protocol for $\mathbf{QK}^\top$ and $\mathsf{Softmax} \cdot \mathbf{V}$ in the decode phase that addresses both inefficiencies: (1) packing multiple query heads into the same ciphertext whenever they share the same $\mathbf{K}$ and $\mathbf{V}$, increasing ciphertext utilization; and (2) making the per-step cost of $\text{Enc}_{\mathbf{K}}$ and $\text{Enc}_{\mathbf{V}}$ independent of N_{kv} by keeping prior rows of $\mathbf{K}^\top$ and $\mathbf{V}$ encrypted across decode steps and encrypting and transmitting only the single new row, incrementally growing the encrypted cache.

5 Nonlinear Operations

Softmax and RMSNorm each evaluate round-heavy subroutines such as exponentiation, division, reciprocal square root, and a row-wise maximum; as we showed in Section 4.2, the rounds these incur dominate PI decode latency. Prior work [15, 39, 56] evaluates these with hybrid HE-OT methods. On the HE side, smooth functions are approximated with iterative methods (Newton's [42], Goldschmidt's [31]), but because these frameworks use shallow HE parameters, a deep circuit must be split into many short sub-circuits separated by interactive noise-refresh rounds. On the OT side, data-dependent branching (row-wise Max, leading-bit detection)

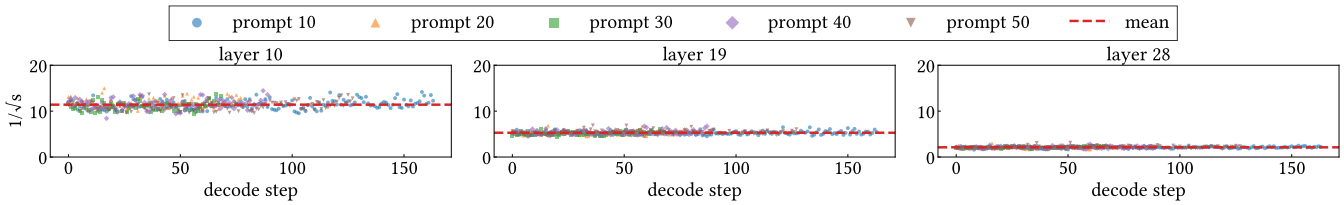


Figure 5: RMSNorm's $\frac{1}{\sqrt{s}}$, where $s = \text{mean}(\vec{x}^2) + \varepsilon$, across decode steps for three sample layers of Llama 3.1 8B on five prompts from GSM8K. Per-layer values are clustered around a precomputed mean. Appendix G shows the same behavior for other layers.

has no natural HE counterpart and falls back to round-heavy protocols. Table 2 breaks down the resulting cost: with two RMSNorm instances per block, Softmax and RMSNorm alone consume *180 rounds of communication per block*.

5.1 Strategy

We attack each source of communication rounds with a strategy tailored to it. First, we exploit knowledge of the inputs. Some rounds exist only because prior protocols must handle inputs of arbitrary magnitude: to approximate the reciprocal square root, for instance, prior work narrows its wide input range dynamically with round-heavy leading-bit detection. We show that RMSNorm can instead be normalized in advance by a constant precomputed from a calibration set, removing the dynamic normalization and its rounds entirely (Section 5.2).

Second, whereas prior work commits to one protocol family exclusively, we match each subtask to the protocol best suited for it. FSS, for instance, is too bandwidth-heavy for evaluating polynomials over many values, but for a small subtask like the row-wise maximum—where OT needs a deep comparison tree with several rounds at each level—its communication stays reasonable (Section 5.3).

Finally, we configure the remaining HE evaluation for decode rather than prefill. Since single-token decode is bound by rounds rather than compute, we adopt heavier HE parameters whose greater multiplicative depth absorbs many multiplications per round—trading local compute, which decode has to spare, for rounds (Section 5.4).

Overall, our modifications lead to 27 rounds for Softmax and 9 rounds for RMSNorm per invocation (Table 2), reducing per-layer communication by 135 rounds (4×).

5.2 Normalization

Goldschmidt's method uses OT-based leading-bit detection to dynamically normalize an input of unknown magnitude into a narrow range so that a short polynomial iteration converges. However, we show that RMSNorm can be normalized by a per-layer constant precomputed from a calibration set instead, and Softmax already normalizes its own inputs.

RMSNorm. RMSNorm computes $\vec{y} = \vec{x} \cdot \frac{1}{\sqrt{s}} \cdot \vec{w}$, where $s = \text{mean}(\vec{x}^2) + \varepsilon$. s spans a wide, input-dependent range, which is why prior work falls back on round-heavy, OT-based leading-bit detection to obtain good, data-dependent initial guesses for Newton's method.

Our key observation is that although s varies widely *across* layers, it is tightly concentrated *within* each layer: for a given layer, s stays close to a layer-specific value across different inputs and decode steps. Figure 5 illustrates this on sample prompts: per-layer $\text{rsqrt}(s)$ remains nearly constant throughout decoding.

We therefore replace the data-dependent normalization with a *precomputed* one. Model owners estimate, once per layer, the mean μ of s over a calibration set and rescale by the constant $c = 1/\mu$, so that $\tilde{s} = c\,s \approx 1$ for typical inputs. Applying the identity $\frac{1}{\sqrt{s}} = \sqrt{c} \cdot \frac{1}{\sqrt{\tilde{s}}}$ then splits RMSNorm into the precomputed constant $\sqrt{c}$ and the data-dependent term $\text{rsqrt}(\tilde{s})$. Unlike s, $\tilde{s}$ is confined to a narrow, known range around 1, over which $\text{rsqrt}(\tilde{s})$ becomes cheap to evaluate securely (Section 5.3).

We validated this technique across different LLMs and datasets and confirmed that the LLM responses match those of the original model exactly, with no accuracy loss (details in Appendix G).

Softmax. Softmax normalizes its inputs by subtracting the row-wise maximum (Max) before exponentiation. This confines each exponential to $(0, 1]$, and hence their sum, the division input, to $[1, N_{\text{kv}}]$. With the range known a priori, dynamic normalization—and its OT rounds—becomes unnecessary.

5.3 Changing Cryptographic Tools

With normalization no longer costing rounds, we turn to the rounds that remain. Techniques such as FSS can eliminate almost all of them, but at a price: generating the FSS keys requires heavy communication. Used wholesale, FSS thus adds more communication latency than the saved rounds recover (see Section 8.3). We instead use FSS only for subtasks that need small keys, where the round savings far outweigh the communication overhead.

RMSNorm's rsqrt is one such subtask: by the time RMSNorm runs, the attention heads are already merged and de-

code processes one token at a time, so each invocation evaluates a single scalar. Our precomputed rescaling (Section 5.2) shrinks the keys further: keys grow with precision and input range, and the rescaling confines the input to a narrow interval around 1. The gate returns shares of rsqrt($\tilde{s}$); RMSNorm's remaining multiplications stay in the native ring (Section 5.4).

The other subtask is the row-wise Max of the attention logits, which reduces to a tree of pairwise comparisons, each needing only the sign of a difference. This 1-bit payload enables specialized optimizations [61] that sharply cut key-generation bandwidth and latency. We also batch key generation for all layers and decode tokens, amortizing its RTT cost. Appendix A details our dealer-free key generation and its optimizations.

5.4 Polynomial Evaluation

The remaining rounds are mostly due to soft bootstraps—interactions where the client refreshes the noise budget—during HE polynomial evaluations. We make them less frequent by using parameters that yield higher multiplicative depth, at the cost of slower compute.

Recall from our latency model (Equation (5)) the compute term of Softmax: $hN \cdot T_{\text{row}}(N_{\text{kv}})$. Slower compute raises the per-row factor $T_{\text{row}}(N_{\text{kv}})$, which is multiplied by $N{=}N_{\text{prefill}}$ tokens during prefill. Since any per-row overhead is amplified by N_{prefill}, prior work favors lighter parameters and achieves faster HE processing. In contrast, during decode $N{=}1$: higher T_{row} is not amplified, so trading slower compute for fewer soft-bootstrap rounds is a net win.

To evaluate the required polynomials, we *lift* the computation from the native ring ($\mathbb{Z}_{2^k}$) into a prime field ($\mathbb{Z}_p$)—recent ring-switching protocols make this practical [25]—and evaluate under CKKS [20] with heavier leveled parameters. We specify the concrete CKKS parameters and the polynomial-evaluation algorithm in Appendix B. Overall, this reduces Softmax to six HE-phase rounds plus three rounds for ring conversion. For RMSNorm, we keep the Beaver triples [8] approach of prior work [39, 47]: during decode RMSNorm operates on a single value, so evaluating it in HE would leave nearly all ciphertext slots idle.

6 Attention Kernels

Autoregressive decode in modern LLMs exhibits two structural properties that existing PI protocols leave unexploited. First, decode grows the cached $\mathbf{K}$ and $\mathbf{V}$ matrices by exactly one row per step while leaving all previously computed rows unchanged. Ideally, the cache should stay encrypted on the server and be updated in place as new rows become available. Second, GQA provides an opportunity to fill the ciphertexts that, as we showed in Section 4.3, are severely underutilized during decode.

It is possible to address both issues by building on slower HE-Matmul baselines that use CRT encoding [69], but this would merely shift the problem to high HE compute latency (see our evaluations in Section 8.4). We instead solve both by building our attention HE-Matmul protocols on top of a baseline HE-Matmul that uses direct polynomial encoding for fast HE compute [39].

6.1 RLWE KV Cache

We introduce an RLWE-level KV cache that the server retains across decode steps; the client sends only an encryption of the newly generated row, which updates the cached ciphertexts in place through HE operations.

For ciphertexts from prior steps to remain valid, *every cached element must map to the same coefficient position and ciphertext at every decode step*—a property determined by how the protocol tiles matrices into RLWE polynomials. Recall from Section 2.3 that a polynomial holds an entire operand only when $d_0 \cdot d_1 \cdot d_2 \leq n$, so matrices of realistic size are *tiled* into sub-blocks that fit this budget. In the decode product $\vec{q}\mathbf{K}^\top$, with $\vec{q} \in \mathbb{R}^{1\times d}$ and the cached $\mathbf{K}^\top \in \mathbb{R}^{d\times N_{\text{kv}}}$, the operands are tiled into $d_0 \times d_1$ and $d_1 \times d_2$ sub-blocks, respectively, each encoded into a single polynomial. This yields

$$b_0 = \lceil 1/d_0 \rceil, \quad b_1 = \lceil d/d_1 \rceil, \quad b_2 = \lceil N_{\text{kv}}/d_2 \rceil$$

blocks per axis: $b_0 \cdot b_1$ polynomials for $\vec{q}$ and $b_1 \cdot b_2$ for $\mathbf{K}^\top$.

Since (d_0, d_1, d_2) determines both the number of ciphertexts communicated and the number of homomorphic multiplications, existing protocols select it with a greedy optimizer that minimizes a weighted cost of both for the shapes *at the current step* [39]. Such a cache-oblivious optimizer undermines ciphertext reuse in two ways:

1. **Unstable positions.** As N_{kv} grows, the optimizer may assign a different d_1, which shifts the coefficient position $k \cdot d_1 + j$ (Equation (2)) of every previously encoded element and invalidates all cached ciphertexts.

2. **Scattered entries.** If the optimizer splits the new data across tiles, each new cache entry spans multiple ciphertexts, requiring multiple delta ciphertexts per step and proportionally diminishing the benefit of caching.

Figure 6 illustrates both failure modes with $n = 10$: the cache $\mathbf{K}^\top$ is 4×4 after prefill ($d = 4$) and grows to 4×5 at the first decode step. At prefill (top), tilings I $(d_1, d_2) = (4, 2)$ and II $(d_1, d_2) = (2, 4)$ cost the same, so the optimizer may pick either; at the first decode step it switches to $(2, 5)$, which fully utilizes the ciphertext ($d_1 \cdot d_2 = n$). If prefill used tiling I, the switch changes d_1, shifting every cached position and forcing full re-encryption (unstable positions). Even under tiling II, where d_1 stays 2, each new column of $\mathbf{K}^\top$ (a new row of $\mathbf{K}$) is split across $\lceil d/d_1 \rceil = 2$ tiles, costing two delta

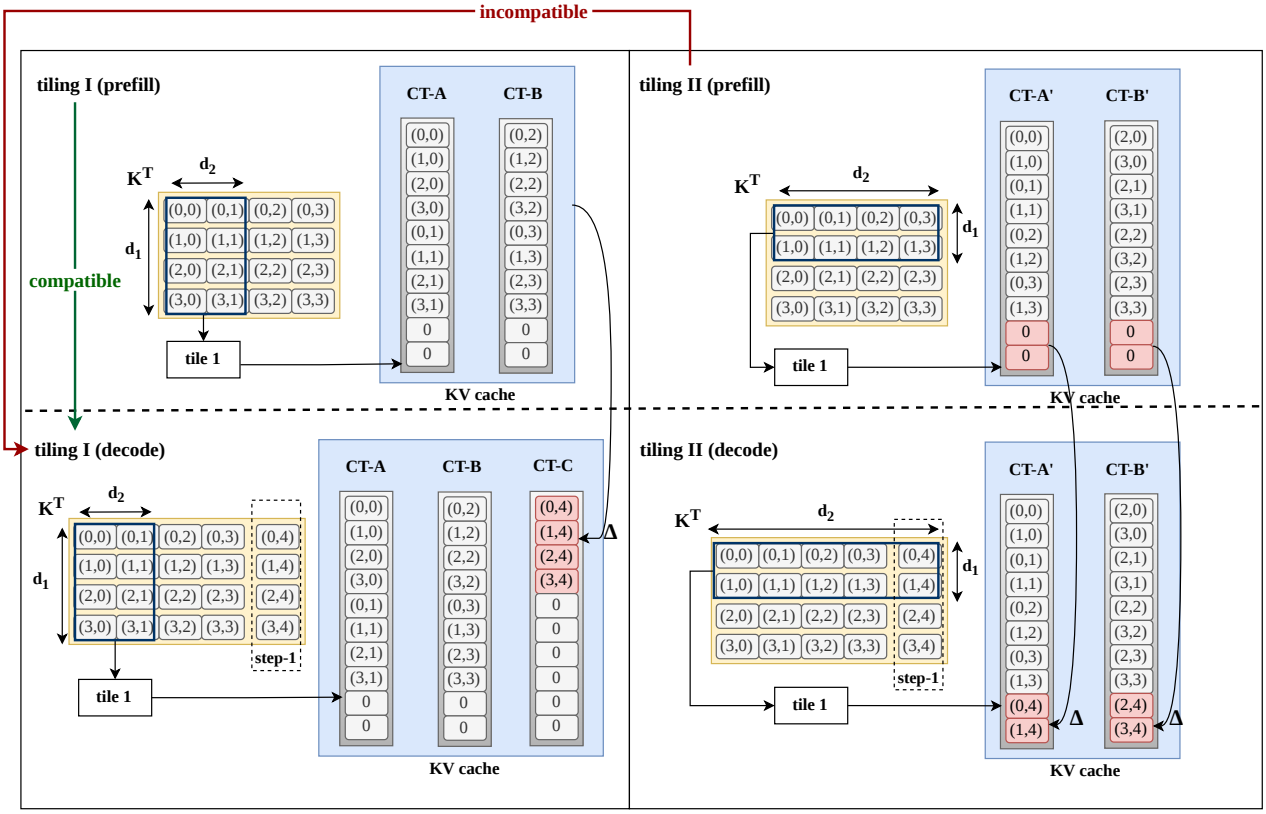


Figure 6: Overview of the RLWE KV cache. The HE-Matmul tiling must be cache-aware: cached elements must keep their ciphertext and coefficient positions across decode steps, and each new row must land in a single ciphertext.

ciphertexts per step (scattered entries). Committing to tiling I from prefill avoids both: with $d_1 = d$, positions never move and each new column lands in one ciphertext, so a single delta ciphertext per step suffices.

Based on these observations, we impose two constraints on the tiling optimizer. First, to keep coefficient positions stable, the optimizer becomes *cache-aware*: the d_1 chosen at cache initialization is pinned for all subsequent decode steps. This is trivial for $\vec{q}\mathbf{K}^\top$, where d_1 tiles the constant head dimension, but for $\mathsf{Softmax} \cdot \mathbf{V}$, d_1 tiles the growing N_{kv}, and pinning it to the initial cache length would waste capacity later. We therefore pad d_1 to its *future maximum* n/d_2 from the start, letting new rows fill the padded positions of the latest block. Second, to avoid scattered entries and the excessive delta ciphertexts they require per update, the optimizer never splits the head dimension ($d_1 = d$ for $\vec{q}\mathbf{K}^\top$, $d_2 = d$ for $\mathsf{Softmax} \cdot \mathbf{V}$), so each decode step updates exactly one ciphertext per head.

6.2 GQA-Aware Packing

HE-Matmuls built on direct polynomial encoding compute one matrix–matrix product per polynomial pair: the coefficient encodings (Equations (1) and (2)) are laid out so that the ring convolution accumulates the inner products of a *single* product $\mathbf{A} \cdot \mathbf{B}$. Two independent products cannot share a polynomial, as their terms would accumulate into the same output coefficients and corrupt each other. Since each attention head performs an independent matrix multiply, multi-head attention is executed as a *batch* of h_q separate HE-Matmuls. This is efficient during prefill, where each head has N query rows to fill its ciphertexts; during decode, however, each head contributes a single row, so every ciphertext in the batch carries one row's worth of useful data while paying full-ciphertext encryption and communication costs.

Rather than switching to CRT encoding, which would slow down the HE compute, we observe that GQA [1] already provides a way to increase ciphertext utilization during decode: the $G = h_q/h$ query heads of a group share the same $\mathbf{K}$ and $\mathbf{V}$ head, so their rows can be treated as a single multi-row matrix. Concretely, we stack the group's query rows into $\mathbf{Q}_g \in \mathbb{R}^{G \times d}$ and compute $\mathbf{Q}_g\mathbf{K}^\top$ as one HE-Matmul against the shared cache; likewise, the group's score rows form a $G \times N_{\text{kv}}$ matrix multiplied by the shared $\mathbf{V}$. The stacked rows occupy the same ciphertexts ($d_0 = G$ instead of 1), raising utilization and reducing both the number of encryptions and the communication volume by up to $G\times$.

7 Methodology

7.1 Baseline PI Systems

We compare against the baselines listed below.

- **Bee and Bee-CS**: BumbleBee [39] is the framework on which we build our system. We also evaluate a modified version of it, Bee-CS, which follows our client–server (CS) setup, in which the client only encrypts and decrypts data while the server performs all the heavy HE computation.
- **SHAFT**: We compare our Softmax protocol against the one proposed by SHAFT [41] (Section 8.3). Its other contributions, such as improved embedding and GeLU protocols, are orthogonal to our study and composable with our system.
- **SIGMA**: SIGMA [33] primarily uses FSS for nonlinear kernels. To remove the trusted dealer assumption and match our setting, we implement the dealer-free version of SIGMA's fully FSS Softmax and RMSNorm and compare them against our protocols (Section 8.3). To make the baseline strong, we optimize the dealer-free key generation as much as possible, as detailed in Appendix C.
- **Cachemir**: Like HERMES, Cachemir [69] keeps an encrypted KV cache on the server. Unlike HERMES, it builds on slower HE-Matmuls that rely on CRT encoding [54, 69]. Cachemir also targets a different setting in which inference is noninteractive and some operations are approximated rather than exact (e.g., Max in Softmax). For a fair comparison, we re-implement Cachemir's HE-Matmul kernels and integrate them into BumbleBee. When comparing its $\mathbf{Q}\cdot\mathbf{K}^\top$ and $\mathsf{Softmax}\cdot\mathbf{V}$ protocols against HERMES in Section 8.4, we exclude the overhead this integration would add, which stems mostly from converting between $\mathbb{Z}_p$ and $\mathbb{Z}_{2^k}$.

We use the default parameters of the baselines unless they need to be changed for correct evaluation of the models we consider (see Appendix C).

7.2 Communication

We model three deployment scenarios, each characterized by its BW and RTT: **1 Gbps, 10 ms**—an enterprise client (e.g., a financial institution or hospital) with a dedicated fiber link to a datacenter [29, 49, 58]; **300 Mbps, 40 ms**—a mobile client on a 5G connection or a fixed client reaching a datacenter in a nearby region [7, 13, 45, 53, 54]; and **200 Mbps, 80 ms**—a client whose nearest datacenter offering the service lies in a distant region, as is common with centralized datacenters [6, 7, 54, 71].

7.3 Machine Learning

We use Llama 3.1 8B Instruct [32, 63] in our evaluations. Our end-to-end experiments solve prompts from *GSM8K* [21], a benchmark of math problems that require multi-step arithmetic reasoning, to full completion; the exact prompts are listed in Appendix D. Our remaining benchmarks additionally draw prompts from the *CNN/DailyMail* news-summarization dataset [36, 51].

7.4 Evaluation Testbed

We run our experiments on VMs on Google Cloud Platform, using two instances in the `northamerica-northeast2` (Toronto) region to represent the client and the server. Unless otherwise stated, each instance has 64 vCPUs, 256 GB of RAM, and 256 GB of SSD storage, and runs Ubuntu 22.04. Finally, we use Linux traffic control (`tc`) to emulate the network conditions introduced earlier.

8 Evaluation

8.1 End-to-End PI

We use three GSM8K prompts, chosen to cover different input-to-output ratios: roughly equal input and output tokens (134 → 112), more output than input (69 → 183), and more input than output (161 → 65). Below we show `GSM8K_train_1648` as an example. The client sends the prompt encrypted; the response is generated entirely under PI and revealed only to the client. The remaining prompts and completions are in Appendix D.

`GSM8K_train_1648` — Input (110 + 24 padding = 134 tokens)

Kenny wants to make sure he does more jumping jacks every week than he did the week before. He recorded that last week he did 324 total jumping jacks. On Saturday of this week, he looks at his records and sees that on Sunday he did 34. On Monday he did 20. On Tuesday he skipped a day. On Wednesday he did 123. On Thursday he did 64. On Friday he did 23. How many does he have to do on Saturday to make sure he beats last week's number?

PI output of HERMES (112 tokens)

Let's calculate the total number of jumping jacks Kenny has done so far this week: 34 (Sunday) + 20 (Monday) + 123 (Wednesday) + 64 (Thursday) + 23 (Friday) = 264. To beat last week's total of 324, Kenny needs to do more than 324 - 264 = 60 jumping jacks on Saturday. Since Kenny wants to make sure he beats last week's number, he should do at least 61 jumping jacks on Saturday. Final answer: 61

Figure 7 reports end-to-end latency and Table 3 reports communication. Decode dominates total latency, and since

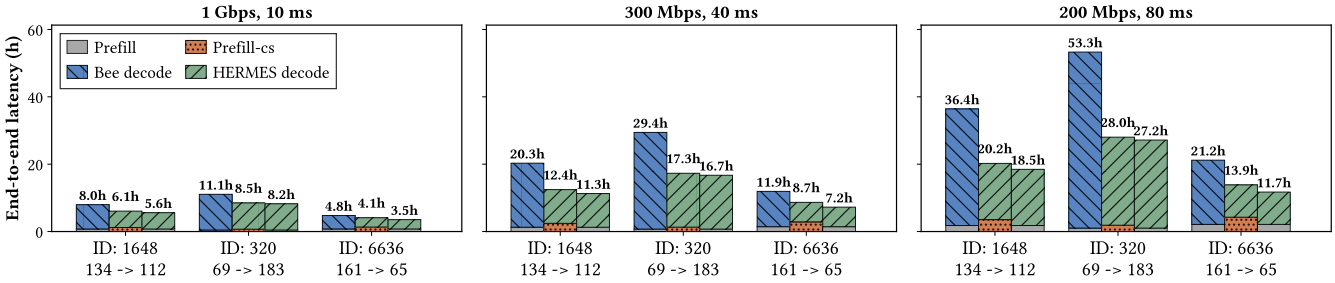


Figure 7: Latency of Llama 3.1 8B Instruct PI on GSM8K prompts. Each ID is the prompt's index in the GSM8K training set. $n \to m$ means the prompt had n tokens and the output m tokens.

Table 3: Communication volume of Llama 3.1 8B Instruct PI on GSM8K prompts (server's view). All values are in GiB.

	Bee		HERMES	
	Sent	Recv	Sent	Recv
GSM8K_train_1648				
Prefill (134 tokens)	49.52	45.96	49.52	45.96
Decode (112 tokens)	135.56	134.51	135.80	113.06
Total	185.08	180.47	185.32	159.02
GSM8K_train_320				
Prefill (69 tokens)	22.17	20.87	22.17	20.87
Decode (183 tokens)	216.58	216.52	266.38	180.06
Total	238.75	237.39	288.55	200.92

Table 4: Communication / rounds breakdown of PI Softmax latency. The values are for one decode step of Llama 3.1 8B.

N_{kv}	Phase	BumbleBee	SIGMA	HERMES
		Comm. (MiB) / Rounds		
64	Keygen	–	1734.5 / 71	327.4 / 33
	Max	6.6 / 1152	3.1 / 384	3.1 / 384
	$\exp(\vec{x})/\sum\exp(\vec{x})$	174.0 / 2336	12.6 / 576	308.7 / 224
	Ring$\leftrightarrow \mathbb{Z}_p$	–	–	1.3 / 64
	Total	**180.6 / 3488**	**1750.2 / 1031**	**640.5 / 705**
512	Keygen	–	11265.7 / 71	989.5 / 33
	Max	54.4 / 1728	25.0 / 576	25.0 / 576
	$\exp(\vec{x})/\sum\exp(\vec{x})$	479.0 / 2336	100.0 / 576	654.2 / 224
	Ring$\leftrightarrow \mathbb{Z}_p$	–	–	10.1 / 64
	Total	**533.4 / 4064**	**11390.7 / 1223**	**1678.8 / 897**

HERMES accelerates precisely this phase, it achieves a 1.4–1.9× end-to-end speedup over Bee, growing as the network degrades, and saving hours of PI latency. We also show a strict client–server variant of prefill (Prefill-CS), which keeps all HE compute on the server. It slightly increases prefill time, but the increase is negligible compared to the savings in decode time; HERMES can run on top of either prefill variant. HERMES also keeps the communication similar to Bee: although its FSS-based protocols add communication, this cost is offset by the savings in attention and by replacing round-intensive protocols with high-depth CKKS circuits.

The rest of this section investigates where the gains come from: we first quantify the TPOT improvement (Section 8.2), then break it down into the nonlinear (Section 8.3) and attention (Section 8.4) kernels.

8.2 TPOT

Figure 8 breaks down the TPOT of HERMES, Bee, and Bee-CS under our three network profiles and increasing KV cache lengths. Even under the best network (1 Gbps, 10 ms), HERMES provides up to a 1.4× speedup over the best baseline (Bee). The speedup grows as network conditions worsen, reaching up to 2.1×.This comes from better attention, which uses GQA to increase ciphertext utilization, and from the lower round counts of our Softmax and RMSNorm protocols. The speedup is much larger over Bee-CS, which, like HERMES and unlike Bee, does not run HE processing on the client's system. Without the client's help, Bee-CS must secret-share the KV cache anew at each decode step, redundant work that HERMES avoids by keeping the cache encrypted on the server. HERMES thus gets the best of both worlds during attention: fixed client–server roles like Bee-CS, with less communication than either baseline thanks to the encrypted cache and GQA-aware packing.

We next examine the gains of our Softmax and RMSNorm protocols in more detail (Section 8.3), as well as how our attention HE-Matmuls achieve their speedup over the baselines (Section 8.4).

8.3 Softmax and RMSNorm

Figure 9 (a) shows the Softmax PI latency of one decode step for Llama 3.1 8B, and Table 4 shows the communication and number of rounds for the smallest and largest KV cache lengths ($N_{kv} = 64$ and 512). Our Softmax protocol outperforms all baselines under most network conditions, and Table 4 explains why: the three protocols occupy very different points in the rounds-versus-communication trade-off space. Bee has the lowest communication, but requires 3488–4064 rounds—roughly 5× more than our 705–897—and since de-

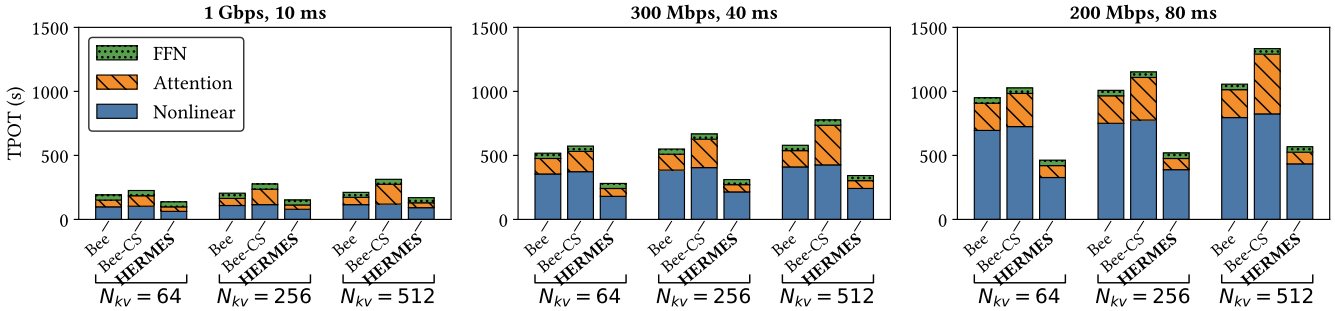


Figure 8: Latency of decoding a single token under different network conditions and KV cache lengths, with prompts drawn from the CNN/DailyMail dataset. "Nonlinear" covers Softmax, RMSNorm, and SiLU; "Attention" and "FFN" cover the Matmul kernels of attention and the FFN, respectively.

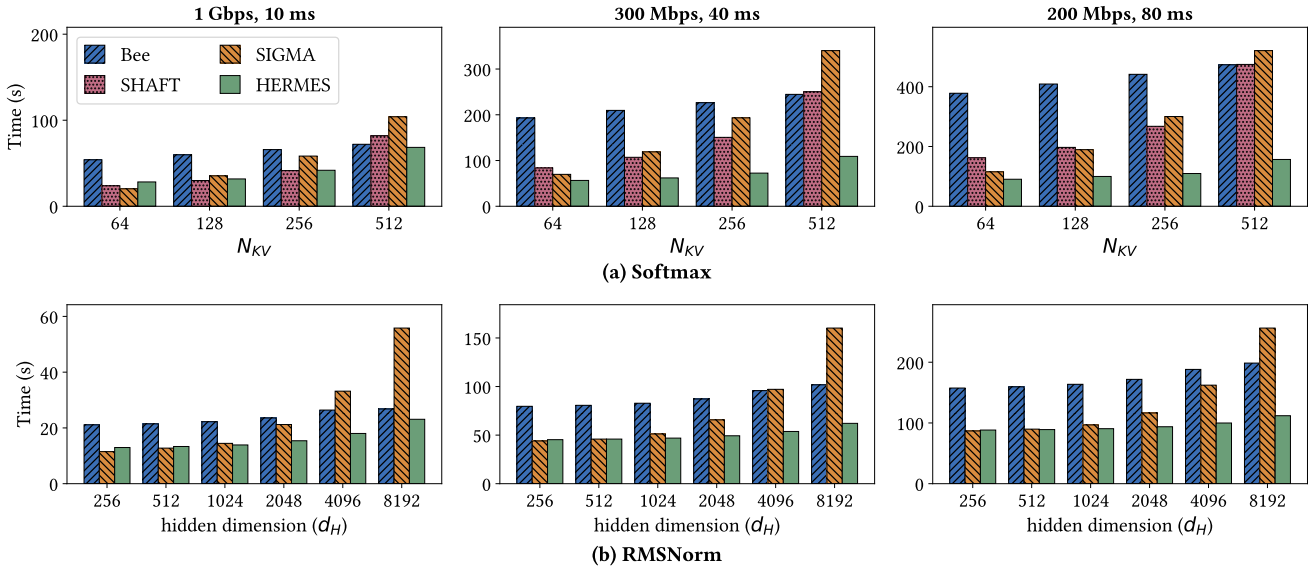


Figure 9: Softmax (a) and RMSNorm (b) latency of one decode step for Llama 3.1 8B under various PI protocols and networks.

code's data volume is small, the round latency dominates. By using FSS only when it is effective and high-depth HE when possible, our protocol cuts the round count while keeping communication moderate, achieving up to 3× lower latency.

SIGMA sits at the opposite extreme: its round count is not as high, but it is heavily bandwidth-bound—its dealer-free FSS key generation transfers 1.7–11 GiB of key material (growing with the KV cache length), which alone takes minutes per token at 200–300 Mbps. Only at 1 Gbps and small N_{kv} does this transfer become cheap enough that SIGMA's lightweight evaluation phase pulls ahead of our HE-based exponentiation.

Figure 9 (b) shows the corresponding results for RMSNorm, which follow the same pattern: Bee is bottlenecked by its round count and SIGMA by its FSS key material, whose size here grows with the hidden dimension rather than the KV cache length. For Llama 3.1 8B's hidden dimension of 4096, HERMES is 1.4×–1.9× faster depending on the baseline and network.

8.4 Linear Kernels of Attention

Figure 10 shows the latency of the attention HE-Matmuls, $\mathbf{Q} \cdot \mathbf{K}^\top$ and $\text{Softmax} \cdot \mathbf{V}$, averaged over five runs. Bee-CS has the highest communication and encryption cost: to avoid running any HE compute on the client's machine, it must re-encrypt and re-transmit the secret-shared KV cache at every decode step. Bee avoids this redundancy by processing the HE-Matmuls on both parties' machines simultaneously, which also makes its HE compute up to 30% faster than HERMES's. However, Bee cannot benefit from GQA, so its communication and encryption costs remain well above HERMES's—savings that more than compensate for the small compute gap and make HERMES's $\mathbf{Q} \cdot \mathbf{K}^\top$ and $\text{Softmax} \cdot \mathbf{V}$ kernels

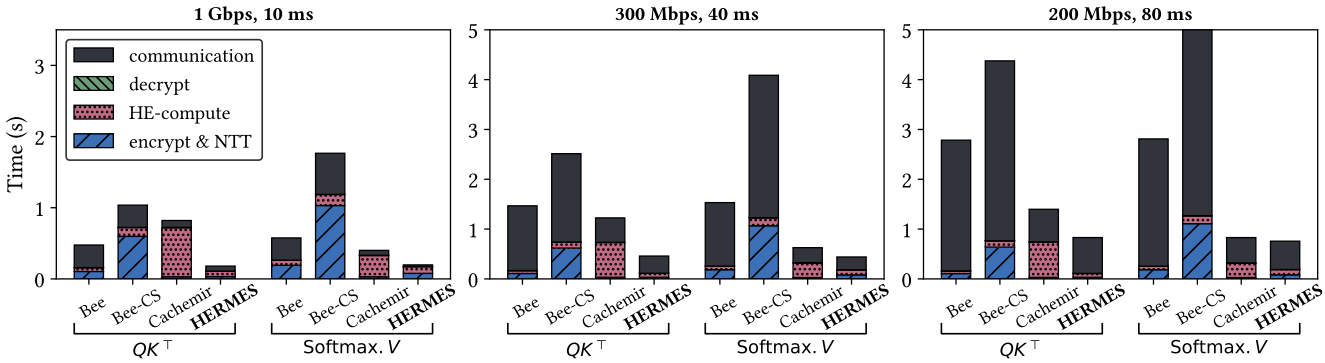


Figure 10: Average latency of Llama 3.1 8B's HE-Matmul executions for $\mathbf{QK}^\top$ and $\mathsf{Softmax} \cdot \mathbf{V}$ with $N_{\text{kv}} = 512$.

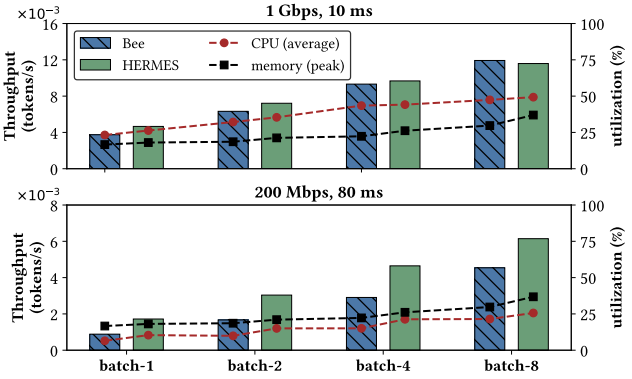


Figure 11: Throughput of batched decode with a KV cache length of 256.

2.6×–3.7× faster than Bee's overall.

Cachemir presents a different trade-off. By using CRT and slot-wise encoding, it achieves highly utilized ciphertexts and low communication, but it pays for this in compute: operating on slot-wise encodings under HE requires expensive rotations, making Cachemir's HE compute the slowest of all baselines. Overall, Cachemir is up to 4.5× and 2.1× slower than HERMES for $\mathbf{Q} \cdot \mathbf{K}^\top$ and $\mathsf{Softmax} \cdot \mathbf{V}$, respectively.

HERMES achieves the best of both worlds: like Bee, it keeps HE-compute low by building its KV cache for rotation-free direct polynomial encoding, yet its communication is far lower—the cache stays encrypted on the server and is updated with a single delta ciphertext per group of heads sharing the same $\mathbf{K}$ and $\mathbf{V}$ head.

8.5 Batched Processing

HERMES is primarily designed for low-latency scenarios. However, in use cases where clients can afford to wait longer (e.g., days instead of hours), batching can help the baselines close the gap by overlapping communication with computation.

Figure 11 reports the throughput (tokens per second) of HERMES and Bee when serving concurrent clients. Batching benefits both systems, but since it hides some of the very overheads HERMES eliminates, the gap between the two narrows as the batch grows—from 1.25× at two clients to parity at eight on the 1 Gbps, 10 ms network. On the slower network, however, HERMES still offers 1.35× higher throughput even at batch size eight. Closing this remaining gap would require batch sizes that force each client to wait a long time for a response.

Figure 11 also reports the resource usage of the two systems. Whenever HERMES delivers higher throughput, it also shows higher average CPU utilization (e.g., 1.5× higher at batch size two on the 200 Mbps, 80 ms network): its cores spend less time idling on communication, confirming that HERMES effectively reduces the communication overhead. The trade-off is memory: HERMES's footprint is higher than Bee's (e.g., 1.1× higher in the same setting), mostly due to storing the precomputed FSS keys.

9 Related Work

PI for convolutional networks. Early PI frameworks targeted convolutional networks [30, 35, 37, 40, 48, 64]: the first generation used CRT-based encoding [40, 48], while later work encoded directly into RLWE polynomial coefficients, avoiding the slow slot permutations [35, 37].

PI for transformers. Recently, a surge of PI frameworks has targeted transformers and LLMs [18, 24, 33, 34, 39, 41, 44, 54, 69, 70]. Iron [34] extended Cheetah's [37] direct polynomial encoding to transformers' Matmuls; BOLT [54] reverted to CRT-based encoding to reduce communication, but BumbleBee [39] instead improved Iron's output packing to avoid BOLT's slot permutations. SIGMA [33] relied fully on FSS, which incurs a high communication overhead unless a trusted dealer is used. SHAFT [41] reduced Softmax's round count, but still requires more rounds than our protocol. It also does not consider RMSNorm's rounds, full decode, or KV caching.

Noninteractive frameworks. NEXUS [70] and Cachemir [69] remove interaction altogether, running inference entirely on the server under HE. However, operations that interactive protocols compute exactly (e.g., row-wise Max) must then be approximated, and it is unclear whether such approximations remain accurate enough for autoregressive decode, where every token must be produced correctly. Furthermore, while Cachemir proposed a KV cache under HE, it is built on CRT techniques and increases HE-Matmul latency compared to frameworks such as BumbleBee.

10 Conclusion

We demonstrated full multi-token response generation under PI, characterized the decode phase, and proposed protocols that reduce communication overhead and manage the KV cache under HE, lowering end-to-end latency by up to 2.1×. Our techniques are best suited to latency-sensitive applications, where a user submits a prompt and expects an answer within a few hours. When higher latency is tolerable and throughput is the main concern, batch processing can instead hide the communication overhead behind computation, although even at small batch sizes our techniques still deliver higher throughput.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, Singapore, December 2023. Association for Computational Linguistics. URL: https://aclanthology.org/2023.emnlp-main.298/, doi:10.18653/v1/2023.emnlp-main.298.

[2] Yoshimasa Akimoto, Kazuto Fukuchi, Youhei Akimoto, and Jun Sakuma. Privformer: Privacy-preserving transformer with mpc. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroSP)*, pages 392–410, 2023. doi:10.1109/EuroSP57164.2023.00031.

[3] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. Homomorphic encryption standard. Cryptology ePrint Archive, Paper 2019/939, 2019. URL: https://eprint.iacr.org/2019/939.

[4] Mutahar Ali, Arjun Arunasalam, and Habiba Farrukh. Understanding users' security and privacy concerns and attitudes towards conversational ai platforms. In *2025 IEEE Symposium on Security and Privacy (SP)*, page 298–316. IEEE, May 2025. URL: http://dx.doi.org/10.1109/SP61157.2025.00241, doi:10.1109/sp61157.2025.00241.

[5] Anthropic. The Claude model family: Claude 3.5 sonnet, claude 3.5 haiku, claude 3 opus, 2024. URL: https://www-cdn.anthropic.com/fed9cc193a14b84131812372d8d5857f8f304c52/Model_Card_Claude_3_Addendum.pdf.

[6] Saikrishna Badrinarayanan, Peihan Miao, Xinyi Shi, Max Tromanhauser, and Ruida Zeng. Updatable private set intersection revisited: Extended functionalities, deletion, and worst-case complexity. In *Advances in Cryptology – ASIACRYPT 2024, Part VI*, volume 15489 of *Lecture Notes in Computer Science*, pages 200–233. Springer Nature Singapore, 2024. URL: https://link.springer.com/chapter/10.1007/978-981-96-0938-3_7.

[7] Sebastian Barros. Solving AI foundational model latency with telco infrastructure, 2025. URL: https://arxiv.org/abs/2504.03708, arXiv:2504.03708.

[8] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology – CRYPTO '91*, pages 420–432, 1992. doi:10.1007/3-540-46766-1_34.

[9] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Advances in Cryptology – EUROCRYPT 2021*, pages 871–900, 2021. doi:10.1007/978-3-030-77886-6_30.

[10] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 1292–1303, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2976749.2978429.

[11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, page 309–325, New York, NY, USA, 2012. Association for Computing Machinery. doi:10.1145/2090236.2090262.

[12] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-lwe and security for key dependent messages. In *Proceedings of the*

31st Annual Conference on Advances in Cryptology, CRYPTO'11, page 505–524, Berlin, Heidelberg, 2011. Springer-Verlag.

[13] Broadband Breakfast. Ookla reports T-Mobile fastest mobile provider, July 2024. Accessed: 2026-08-14. Summarizes Ookla Speedtest Connectivity Report, January–June 2024. URL: https://broadbandbreakfast.com/ookla-reports-t-mobile-fastest-mobile-provider/.

[14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL: https://arxiv.org/abs/2005.14165, arXiv:2005.14165.

[15] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shardul Tripathi. Ezpc: Programmable and efficient secure two-party computation for machine learning. In *2019 IEEE European Symposium on Security and Privacy (EuroSP)*, pages 496–511, 2019. doi:10.1109/EuroSP.2019.00043.

[16] Hao Chen, Kim Laine, and Rachel Player. Simple encrypted arithmetic library - SEAL v2.1. Cryptology ePrint Archive, Paper 2017/224, 2017. URL: https://eprint.iacr.org/2017/224.

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[18] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. The-x: Privacy-preserving transformer inference with homomorphic encryption, 2022. URL: https://arxiv.org/abs/2206.00216, arXiv:2206.00216.

[19] Yuntian Chen, Xianjia Meng, Zhiying Shi, Zhiyuan Ning, and Jingzhi Lin. Securetlm: Private inference for transformer-based large model with mpc. *Inf. Sci.*, 667(C), May 2024. doi:10.1016/j.ins.2024.120429.

[20] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer International Publishing, 2017. doi:10.1007/978-3-319-70694-8_15.

[21] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL: https://arxiv.org/abs/1810.04805, arXiv:1810.04805.

[23] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 523–535, 2017. doi:10.1145/3133956.3133967.

[24] Ye Dong, Wen jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. Puma: Secure inference of llama-7b in five minutes, 2023. URL: https://arxiv.org/abs/2307.12533, arXiv:2307.12533.

[25] Ye Dong, Wen-jie Lu, Xiaoyang Hou, Kang Yang, and Jian Liu. M&m: Secure two-party machine learning through modulus conversion and mixed-mode protocols. *IEEE Transactions on Dependable and Secure Computing*, 23(2):4121–4135, 2026. doi:10.1109/TDSC.2025.3643323.

[26] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words:

Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[27] T. Elgamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985. doi:10.1109/TIT.1985.1057074.

[28] Gemini Team et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024. URL: https://arxiv.org/abs/2403.05530, arXiv:2403.05530.

[29] Federal Communications Commission. Measuring fixed broadband – thirteenth report. Technical report, FCC, 2024. Accessed: 2026-04-10. URL: https://www.fcc.gov/reports-research/reports/measuring-broadband-america/measuring-fixed-broadband-thirteenth-report.

[30] Karthik Garimella, Zahra Ghodsi, Nandan Kumar Jha, Siddharth Garg, and Brandon Reagen. Characterizing and optimizing end-to-end systems for private inference. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, page 89–104, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3582016.3582065.

[31] Robert E. Goldschmidt. Applications of division by convergence. Master's thesis, Massachusetts Institute of Technology, 1964.

[32] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet,

Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL: https://arxiv.org/abs/2407.21783, arXiv:2407.21783.

[33] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. SIGMA: Secure GPT inference with function secret sharing. *Proceedings on Privacy Enhancing Technologies*, 2024(4):61–79, 2024. doi:10.56553/popets-2024-0107.

[34] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 15718–15731. Curran Associates, Inc., 2022. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/64e2449d74f84e5b1a5c96ba7b3d308e-Paper-Conference.pdf.

[35] Jiaxing He, Kang Yang, Guofeng Tang, Zhangjie Huang, Li Lin, Changzheng Wei, Ying Yan, and Wei Wang. Rhombus: Fast homomorphic matrix-vector multiplication for secure two-party inference. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 2490–2504, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690281.

[36] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[37] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure Two-Party deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 809–826, Boston, MA, August 2022. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong.

[38] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, pages 145–161, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[39] Wen jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. BumbleBee: Secure two-party inference framework for large transformers. Cryptology ePrint Archive, Paper 2023/1678, 2023. URL: https://eprint.iacr.org/2023/1678.

[40] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. Gazelle: a low latency framework for secure neural network inference. In *Proceedings of the 27th USENIX Conference on Security Symposium*, SEC'18, page 1651–1668, USA, 2018. USENIX Association.

[41] Andes Y. L. Kei and Sherman S. M. Chow. SHAFT: Secure, handy, accurate, and fast transformer inference. In *NDSS*, 2025.

[42] David Kincaid and E. Ward Cheney. *Numerical Analysis: Mathematics of Scientific Computing*. American Mathematical Society, 3rd edition, 2002.

[43] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 19274–19286, 2023. URL: https://proceedings.mlr.press/v202/leviathan23a.html.

[44] Zhengyi Li, Kang Yang, Jin Tan, Wen jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, and Jingwen Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[45] Reiner Ludwig. Who cares about latency in 5G? Ericsson blog, August 2022. Accessed: 2026-08-14. URL: https://www.ericsson.com/en/blog/2022/8/who-cares-about-latency-in-5g.

[46] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Proceedings of the 29th Annual International Conference on Theory and Applications of Cryptographic Techniques*, EUROCRYPT'10, page 1–23, Berlin, Heidelberg, 2010. Springer-Verlag. doi:10.1007/978-3-642-13190-5_1.

[47] Junming Ma, Yancheng Zheng, Jun Feng, Derun Zhao, Haoqi Wu, Wenjing Fang, Jin Tan, Chaofan Yu, Benyu Zhang, and Lei Wang. SecretFlow-SPU: A performant and User-Friendly framework for Privacy-Preserving machine learning. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 17–33, Boston, MA, July 2023. USENIX Association. URL: https://www.usenix.org/conference/atc23/presentation/ma.

[48] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: a cryptographic inference service for neural networks. In *Proceedings of the 29th USENIX Conference on Security Symposium*, SEC'20, USA, 2020. USENIX Association.

[49] Ricky K. P. Mok, Hongyu Zou, Rui Yang, Tom Koch, Ethan Katz-Bassett, and K C Claffy. Measuring the network performance of google cloud platform. In *Proceedings of the 21st ACM Internet Measurement Conference*, IMC '21, page 54–61, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3487552.3487862.

[50] Michael Naehrig, Kristin Lauter, and Vinod Vaikuntanathan. Can homomorphic encryption be practical? In *Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop*, CCSW '11, page 113–124, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/2046660.2046682.

[51] Ramesh Nallapati, Bowen Zhou, Cícero dos Santos, Çağlar Gülçehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence rnns and beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, pages 280–290, 2016.

[52] Kevin Nam, Youyeon Joo, Seungjin Ha, and Yunheung Paek. Slothe: lazy approximation of non-arithmetic neural network functions over encrypted data. In *Proceedings of the 34th USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association.

[53] Ookla. United states speedtest connectivity report, H1 2025. Technical report, 2025. Accessed: 2026-04-10. URL: https://www.ookla.com/research/reports/united-states-speedtest-connectivity-report-h1-2025.

[54] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4753–4771, 2024. URL: https://api.semanticscholar.org/CorpusID:266178755.

[55] Michael Rabin. How to exchange secrets with oblivious transfer. *IACR Cryptology ePrint Archive*, 2005:187, 01 2005.

[56] Deevashwer Rathee, Anwesh Bhattacharya, Rahul Sharma, Divya Gupta, Nishanth Chandran, and Aseem Rastogi. Secfloat: Accurate floating-point meets secure 2-party computation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 576–595, 2022. doi:10.1109/SP46214.2022.9833697.

[57] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. Cryptflow2: Practical 2-party secure inference. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 325–342, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3372297.3417274.

[58] RS Inc. Bandwidth vs. latency: We chatted with an internet connectivity expert, 2025. Accessed: 2026-04-10. URL: https://www.rsinc.com/bandwidth-vs-latency-we-chatted-with-an-internet-connectivity-expert.php.

[59] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. F1: A fast and programmable accelerator for fully homomorphic encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 238–252, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3466752.3480070.

[60] Luohe Shi, Hongyi Zhang, Yao Yao, Zuchao Li, and Hai Zhao. Keep the cost down: A review on methods to optimize llm' s kv-cache consumption, 2024. URL: https://arxiv.org/abs/2407.18003, arXiv:2407.18003.

[61] Kyle Storrier, Adithya Vadapalli, Allan Lyons, and Ryan Henry. Grotto: Screaming fast $(2+1)$-PC for $\mathbb{Z}_{2^n}$ via $(2,2)$-DPFs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023. doi:10.1145/3576915.3623147.

[62] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagic, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozińska, Dustin Herbison, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshev, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucińska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Ju yeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjoesund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, Lilly McNealus, Livio Baldini Soares, Logan Kilpatrick, Lucas Dixon, Luciano Martins, Machel Reid, Manvinder Singh, Mark Iverson, Martin Görner, Mat Velloso, Mateo Wirth, Matt Davidow, Matt Miller, Matthew Rahtz, Matthew Watson, Meg Risdal, Mehran Kazemi, Michael Moynihan, Ming Zhang, Minsuk Kahng, Minwoo Park, Mofi Rahman, Mohit Khatwani, Natalie Dao, Nenshad Bardoliwalla, Nesh Devanathan, Neta Dumai, Nilay Chauhan, Oscar Wahltinez, Pankil Botarda, Parker Barnes, Paul Barham, Paul Michel, Pengchong Jin, Petko Georgiev, Phil Culliton, Pradeep Kuppala, Ramona Comanescu, Ramona Merhej, Reena Jana, Reza Ardeshir Rokni, Rishabh Agarwal, Ryan Mullins, Samaneh Saadat, Sara Mc

Carthy, Sarah Cogan, Sarah Perrin, Sébastien M. R. Arnold, Sebastian Krause, Shengyang Dai, Shruti Garg, Shruti Sheth, Sue Ronstrom, Susan Chan, Timothy Jordan, Ting Yu, Tom Eccles, Tom Hennigan, Tomas Kocisky, Tulsee Doshi, Vihan Jain, Vikas Yadav, Vilobh Meshram, Vishal Dharmadhikari, Warren Barkley, Wei Wei, Wenming Ye, Woohyun Han, Woosuk Kwon, Xiang Xu, Zhe Shen, Zhitao Gong, Zichuan Wei, Victor Cotruta, Phoebe Kirk, Anand Rao, Minh Giang, Ludovic Peran, Tris Warkentin, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, D. Sculley, Jeanine Banks, Anca Dragan, Slav Petrov, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Sebastian Borgeaud, Noah Fiedel, Armand Joulin, Kathleen Kenealy, Robert Dadashi, and Alek Andreev. Gemma 2: Improving open language models at a practical size, 2024. URL: https://arxiv.org/abs/2408.00118, arXiv:2408.00118.

[63] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL: https://arxiv.org/abs/2302.13971, arXiv:2302.13971.

[64] McKenzie van der Hagen and Brandon Lucia. Client-optimized algorithms and acceleration for encrypted compute offloading. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, page 683–696, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3503222.3507737.

[65] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL: https://arxiv.org/abs/1706.03762, arXiv:1706.03762.

[66] Pengzhi Xing, Hongwei Li, Meng Hao, Hanxiao Chen, Jia Hu, and Dongxiao Liu. Distributed function secret sharing and applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi:10.14722/ndss.2025.242233.

[67] Tianshi Xu, Wen-jie Lu, Jiangrui Yu, Yi Chen, Chenqi Lin, Runsheng Wang, and Meng Li. Breaking the layer barrier: remodeling private transformer inference with hybrid ckks and mpc. In *Proceedings of the 34th USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association.

[68] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated ot with small communication. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 1607–1626, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3372297.3417276.

[69] Ye Yu, Yifan Zhou, Yi Chen, Pedro Soto, Wenjie Xiong, and Meng Li. Cachemir: Fully homomorphic encrypted inference of generative large language model with kv cache, 2026. URL: https://arxiv.org/abs/2602.11470, arXiv:2602.11470.

[70] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. Cryptology ePrint Archive, Paper 2024/136, 2024. URL: https://eprint.iacr.org/2024/136.

[71] Qiao Zhang, Minghui Xu, Tingchuang Zhang, and Xiuzhen Cheng. Almost-free queue jumping for prior inputs in private neural inference, 2026. URL: https://arxiv.org/abs/2603.12946, arXiv:2603.12946.

[72] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, July 2024. USENIX Association. URL: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

Appendix A: FSS-Based Row-Wise Max

This appendix expands on Section 5.3: how our FSS-based row-wise Max specializes the generic FSS design space, and why each optimization below is unavailable to a *generic* FSS gate [9, 10]—one that must carry an arbitrary multi-bit payload through the key tree and produce arithmetic output shares—or to distributed generation for data-dependent secret points, as in ORAM-style uses of FSS [23]. Our keys are generated dealer-free by the two parties themselves, as a distributed tree expansion in the style of Doerner and shelat [23] run over Ferret OT [68].

Single-bit payloads. Because every gate outputs one bit, we generate distributed *point* functions (DPFs) rather than the costlier distributed *comparison* functions (DCFs) [9, 10]: the key tree's control bits are already XOR-shares of "the secret point lies in this subtree," so a comparison is a local parity walk over them [61], and the per-level value-correction words—and the interactive shared-algebra machinery that

computes them on shares [66]—disappear from keygen entirely. One-bit outputs also pack 128 leaves into a single 128-bit word, letting the tree terminate 7 levels early [10] ($\sim$500$\times$ less keygen compute and memory, and 7 fewer exchanges during distributed generation), with the final leaf correction reducing to XOR-shares of a one-hot bit vector obtainable from a few cheap correlated-OT rounds. Finally, Boolean outputs let the online selection step consume lightweight bit-level daBit correlations instead of ring Beaver triples [8], removing OLE from keygen altogether. None of this survives a generic payload: a control bit cannot carry a multi-bit group element, ℓ-bit outputs pack only $128/\ell$ leaves per word, a payload-carrying leaf becomes an arithmetic object requiring far heavier OT, and arithmetic output shares that feed subsequent multiplications need genuine ring triples.

Public endpoints in mask space. For an opened masked input $\hat{x} = x + r$, each sign-region test $\mathbf{1}\{a \le \hat{x} - r \le b\}$ (all arithmetic mod 2^k) rewrites as $r \in [\hat{x} - b, \hat{x} - a]$: an interval test on the key's secret point whose endpoints are *public* at evaluation time [61], with the wrap-around case handled by one party publicly flipping its output bit. A single tree therefore serves all sign regions with no per-region key state and no wrap-correction phase. A generic interval-containment gate [9] cannot do this: its arithmetic output shares require secret-shared wrap corrections, generated per region with millionaires', equality, and B2A subprotocols [57].

Self-chosen secret points. The hidden point of each DPF is a mask the parties invent, not a data value. They can thus sample it directly as XOR shares—the form DPF keygen consumes natively [23]—and derive the additive shares needed by the online opens with a single packed B2A, deleting the additive-to-XOR conversion cascade of millionaires' instances [57] that a data-dependent secret point would force. More importantly, since no key depends on any logit, generation is fully input-independent: it moves off the critical path and batches arbitrarily, whereas a generic secure evaluation is inherently online because its inputs do not yet exist.

Statically known workload. Every OT that the level-by-level generation consumes is a function of upfront-known share bits or fresh randomness, so the correlations for *all* levels of *all* keys are produced by a handful of one-time batched calls, leaving only two packed exchanges per level. The sign tree and its companion carry tree—which recovers the inter-share carry that makes the comparison bit-exact—are always generated in pairs and run in lockstep, sharing every round, so the round budget is independent of batch size; and because correction words are public key material that nobody needs until evaluation, their opens ride the next level's exchange for free. Generic interactive 2PC enjoys none of these: OT choice bits depend on values computed mid-protocol and cannot be hoisted, and intermediate opens sit on the critical path.

Appendix B: HE Parameters and Evaluation Algorithm

This appendix details the low-round HE evaluation of Section 5.4, which we use for Softmax after it is normalized by the row-wise Max.

CKKS instantiation. The HE evaluation of Softmax proceeds in four phases: (1) exponentiation of the shifted logits, (2) a polynomial seed for the reciprocal of the exponential sum, (3) Newton refinement of that seed, and (4) the final element-wise product $\exp(x_i) \cdot S^{-1}$.

We use the same CKKS instantiation for Phases 1–3: ring dimension $n = 2^{14}$, which provides 8192 usable SIMD slots that can pack $\lfloor \frac{8192}{N_{\text{kv}}} \rfloor$ attention heads per ciphertext; a coefficient-modulus chain of nine primes ($\{50, 48^{(\times 7)}, 50\}$ bits); and scaling factor $\Delta = 2^{48}$. This configuration provides seven usable multiplicative levels in the SEAL library [16], while its 436-bit total modulus satisfies 128-bit security at this ring dimension according to the HE standard [3].

The levels are consumed by the ciphertext squarings of the exponential, the Horner evaluation of the reciprocal seed, and the two multiplications of each Newton iteration (detailed below); each phase fits in at most six levels, leaving one level of noise headroom.

Phase 4 performs only a single multiplication, so we switch to a more compact context: $n = 2^{13}$, primes $\{55, 50, 50, 55\}$ (210-bit modulus for 128-bit security at $n = 2^{13}$), and $\Delta = 2^{50}$.

Softmax evaluation. Algorithm 1 lists the full protocol, and Table 5 summarizes its four phases, the level consumption of each segment, and where the soft-bootstraps occur. The key point is that each phase is shaped to fit within the available multiplicative depth, with an interactive refresh only at the segment boundaries. Our Softmax completes in six rounds, independent of N_{kv}.

In more detail, Phase 1 evaluates the exponential with the composition approximation $\exp(x) \approx (1 + x/2^{10})^{2^{10}}$, which costs one plaintext multiplication followed by ten ciphertext squarings. Because the row maximum is computed *exactly* via the FSS protocol of Section 5.3, the shifted logits satisfy $x_i - \max_j x_j \le 0$, so every $\exp(x_i)$ lies in $(0, 1]$ and the per-head sum $S = \sum_i \exp(x_i)$ lies in $[1, N_{\text{kv}}]$. This tight range makes Phase 2 inexpensive: after normalizing to $\tilde{s} = S/N_{\text{kv}} \in [1/N_{\text{kv}}, 1]$, we approximate $1/\tilde{s}$ with a degree-5 polynomial interpolating $1/x$ at Chebyshev nodes. Its evaluation uses exactly six multiplicative levels: one for the affine map $S \mapsto \tilde{s}$ and five for Horner's rule.

Phase 3 refines that seed with three Newton iterations, $y \leftarrow y(2 - \tilde{s}y)$, each costing two multiplicative levels and squaring the relative error; the last iteration additionally rescales by $1/N_{\text{kv}}$ to undo the normalization and produce shares of S^{-1}. Finally, Phase 4 assembles the output in the smaller $n = 2^{13}$ context: the shares of $\exp(x_i)$ from Phase 1

Table 5: High-level overview of the Softmax evaluation with CKKS after it is normalized by the row-wise Max. Every segment ends with one mask–decrypt round: intra-phase rows are *soft-bootstraps* that refresh the ciphertext to the top level, and the remaining rows are the phase outputs (which refresh levels as a side effect). Total: 6 rounds.

Segment	Homomorphic computation	Levels consumed	Round ends with
Phase 1a	prescale $x/2^{10}$ (1); 5× squaring (5)	6	soft-bootstrap
Phase 1b	5× squaring (5)	5	shares of $\exp(x_i)$
local: $\langle S\rangle = \sum_i \langle \exp(x_i)\rangle$		—	no communication
Phase 2	affine map $S \mapsto \tilde{s}$ (1); Horner deg. 5 (5)	6	shares of $y_0 \approx N_{\text{kv}}/S$
Phase 3a	normalize $\tilde{s}$ (1); 2× Newton (4)	5	soft-bootstrap
Phase 3b	normalize $\tilde{s}$ (1); 1× Newton (2); $\times 1/N_{\text{kv}}$ (1)	4	shares of S^{-1}
Phase 4	$\exp(x_i)\cdot S^{-1}$ (1, small context)	1	$\mathbb{Z}_p$ shares of $\text{softmax}(x)$

and the broadcast shares of S^{-1} from Phase 3 are re-encrypted, multiplied element-wise, and then masked and decrypted into fresh $\mathbb{Z}_p$ shares of $\text{softmax}(x)_i = \exp(x_i)\cdot S^{-1}$ at fixed-point scale 2^f, ready for the ring conversion that feeds the subsequent $\text{Softmax}\cdot\mathbf{V}$ product.

Soft-bootstrapping. The full pipeline (ten squarings, the degree-5 seed, and three Newton steps) exceeds the seven-level budget of the modulus chain, but instead of employing full CKKS bootstrapping—whose cost would dwarf the computation itself—we exploit the two-party setting and refresh levels *interactively*. Whenever the remaining level budget cannot hold the next operation, the evaluator subtracts a uniformly random mask from the ciphertext and sends it to the encryptor, who decrypts and keeps the result as its additive share while the evaluator keeps the mask; the shared value then re-enters the next segment as a fresh, top-level ciphertext. This *soft bootstrap* costs a single communication round and keeps every intermediate value secret-shared.

Appendix C: Baseline Setup

This appendix details the parameter choices and implementation optimizations used for the baselines in Section 7.1.

Bee. To reduce the large memory requirement of the embedding and FFN layers of Llama 3.1 8B, we partition the corresponding HE-Matmuls into chunks and stream them, making BumbleBee practical to run on today's hardware. Finally, we increase BumbleBee's default number of fractional bits of the fixed-point representation to 20 and its default number of iterations for approximating exponentials to 10, which we found necessary to produce correct tokens with Llama 3.1 8B.

SHAFT. SHAFT clips the logits into the range (α,β) and uses t ODE iterations to approximate the Softmax probabilities. We found its default parameters, $(\alpha=-3,\beta=12)$ and $t=16$, insufficient to produce correct tokens with Llama 3.1 8B, and increased them to $(\alpha=-20,\beta=32)$ and $t=64$.

Table 6: Prompts used in the end-to-end experiments. Input tokens = question tokens + 24 tokens of chat-template padding.

GSM8K ID	Question	Padding	Input	Output	Answer
`train_320`	45	24	69	183	80 ✓
`train_1648`	110	24	134	112	61 ✓
`train_6636`	137	24	161	65	1250 ✓

SIGMA. To make this baseline as strong as possible, we equip our implementation with all applicable optimizations, including the key-size and key-generation improvements of Grotto [61] over generic FSS gates [9, 10] and batching key generation across all layers and decode steps to amortize its round cost.

Appendix D: Evaluation Prompts

Our end-to-end experiments use three problems from the GSM8K training split. Each question is wrapped in the Llama 3.1 Instruct chat template as shown below:

Chat template

```
<|begin_of_text|><|start_header_id|>system
<|end_header_id|>
End with: Final answer: <number><|eot_id|>
<|start_header_id|>user<|end_header_id|>
<question><|eot_id|><|start_header_id|>assistant
<|end_header_id|>
```

The template adds a fixed padding overhead of 24 tokens on top of the raw question tokens. Outputs are generated greedily with Llama 3.1 8B Instruct, and the output token counts include the final `<|eot_id|>` stop token. All three completions end with the correct final answer. We identify the prompts by their line index in the split and summarize these IDs, token counts, and final answers in Table 6. We also show the exact input prompts and outputs produced by HERMES below.

GSM8K_train_320 — Input (45 + 24 padding = 69 tokens)

Austin bought his seven friends each a robot. Each robot costs $8.75. He was charged $7.22 total for tax. He left with $11.53 in change. How much did Austin start with?

PI output of HERMES (183 tokens)

To find out how much Austin started with, we need to calculate the total cost of the robots and add the tax. The cost of each robot is $8.75, and Austin bought 7 robots, so the total cost of the robots is: 7 x $8.75 = $61.25. Adding the tax, the total amount Austin paid is: $61.25 + $7.22 = $68.47. Since Austin left with $11.53 in change, we can set up an equation to represent the situation: Initial amount - Total amount paid = Change; Initial amount - $68.47 = $11.53. To solve for the initial amount, we can add $68.47 to both sides of the equation: Initial amount = $68.47 + $11.53; Initial amount = $80.00. Final answer: $80.00

GSM8K_train_1648 — Input (110 + 24 padding = 134 tokens)

Kenny wants to make sure he does more jumping jacks every week than he did the week before. He recorded that last week he did 324 total jumping jacks. On Saturday of this week, he looks at his records and sees that on Sunday he did 34. On Monday he did 20. On Tuesday he skipped a day. On Wednesday he did 123. On Thursday he did 64. On Friday he did 23. How many does he have to do on Saturday to make sure he beats last week's number?

PI output of HERMES (112 tokens)

Let's calculate the total number of jumping jacks Kenny has done so far this week: 34 (Sunday) + 20 (Monday) + 123 (Wednesday) + 64 (Thursday) + 23 (Friday) = 264. To beat last week's total of 324, Kenny needs to do more than 324 - 264 = 60 jumping jacks on Saturday. Since Kenny wants to make sure he beats last week's number, he should do at least 61 jumping jacks on Saturday. Final answer: 61

GSM8K_train_6636 — Input (137 + 24 padding = 161 tokens)

One dandelion seed floated onto Mrs. Middleton's lawn, landed on good soil, and began to grow. After one month, the dandelion plant had grown to full size and developed a flower. At the end of the second month, the flower turned to 50 fluffy, white seeds, which blew onto the lawn. Only half of these seeds landed onto good soil, germinated, and began to grow, just as the first seed had done. If each of these new seedling plants produces the same number of seeds in the same amount of time as the original plant, what is the total number of seeds that these newest plants will produce in two months' time?

PI output of HERMES (65 tokens)

The first seed produced 50 seeds. Half of these seeds, or 25, germinated and grew. Each of these 25 plants will produce 50 seeds. Therefore, the total number of seeds produced by the newest plants will be 25 x 50 = 1250.
Final answer: 1250

Appendix E: Cryptographic Tools

Two-party additive secret sharing. We work in a two-party computation between an *encryptor* (party 0) and an *evaluator* (party 1). A real value $\tilde{x} \in \mathbb{R}$ is first quantized to a fixed-point integer $x = \lfloor \tilde{x} \cdot 2^f \rceil \in \mathbb{Z}_{2^k}$ with precision f bits, then additively secret-shared as $\langle x \rangle_0, \langle x \rangle_1 \in \mathbb{Z}_{2^k}$ with $\langle x \rangle_0 + \langle x \rangle_1 \equiv x \pmod{2^k}$. Each share is uniformly random and by itself reveals nothing about x.

Oblivious transfer. A 1-out-of-2 OT [38, 55] lets a receiver, holding a choice bit c, learn one of two sender messages (m_0, m_1) without the sender learning c or the receiver learning m_{1-c}. The Correlated OT (COT) variant reduces the bandwidth when the sender messages are related. We use the Ferret protocol [68] for low-bandwidth COT. Nonlinear primitives such as comparisons and multiplexers reduce to many parallel (C)OT instances, and often require several rounds of communication. OT protocols are more efficient over power-of-two rings $\mathbb{Z}_{2^k}$ [37, 56], and as such $\mathbb{Z}_{2^k}$ is often used as the native ring that secret shares live in [39, 54].

Function secret sharing. FSS [10] splits a function f into two data-independent keys k_0, k_1 that reveal nothing about f in isolation. Once keys are generated using OT protocols or a trusted dealer, each party can locally evaluate its key on a common input x to obtain an additive share of $f(x)$ with little interaction.

Homomorphic encryption. Homomorphic encryption (HE) [11, 27] lets a party holding a ciphertext of x compute a ciphertext of $F(x)$ without ever observing x in the clear. Like most prior work [34, 37, 39, 48, 54], we use schemes built on Ring-Learning-with-Errors (RLWE) [12, 20, 46, 50].

An RLWE scheme is parameterized by a plaintext modulus p and a much larger ciphertext modulus $q \gg p$. A message m is first encoded into a plaintext polynomial $\mathrm{RLWE}(m) \in \mathcal{R}_p$, which a keypair $(\mathrm{sk}, \mathrm{pk})$ encrypts into a ciphertext $[\![\mathrm{RLWE}(m)]\!] \in \mathcal{R}_q^2$; decryption with sk recovers $\mathrm{RLWE}(m)$ which can then be decoded into m. Each operation on $[\![\mathrm{RLWE}(m)]\!]$ enlarges a small *noise* term, and decryption succeeds only while this noise stays under a budget controlled by q/p.

Appendix F: HE-Matmul

We adopt and extend the HE-Matmul protocol from BumbleBee [39], which computes the matrix product $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$ in a single round of communication. The encryptor encrypts $\mathbf{A} \in \mathbb{R}^{d_0 \times d_1}$ into $[\![\mathrm{RLWE}(\mathbf{A})]\!]$ and sends it to the

evaluator; the evaluator encodes $\mathbf{B} \in \mathbb{R}^{d_1 \times d_2}$ as a plaintext polynomial $\mathrm{RLWE}(\mathbf{B})$ and homomorphically computes $[\![\mathrm{RLWE}(\mathbf{C})]\!] = [\![\mathrm{RLWE}(\mathbf{A})]\!] \boxtimes \mathrm{RLWE}(\mathbf{B})$. The protocol operates in the polynomial ring $\mathcal{R}_{q,n}$; for ease of explanation we assume $d_0 \cdot d_1 \cdot d_2 \le n$, so the result fits in one ciphertext and no tiling is needed.

The protocol exploits the fact that multiplication in $\mathcal{R}_{q,n}$ is a convolution of coefficient vectors: products of coefficients accumulate wherever their indices sum to the same value, with a sign flip when that sum wraps past n (since $X^n \equiv -1$). A convolution sums products—exactly the structure of a dot product—so by placing matrix entries at well-chosen coefficients, a single polynomial multiplication evaluates all $d_0 d_2$ dot products between rows of $\mathbf{A}$ and columns of $\mathbf{B}$ at once. Concretely, writing i, j, k for the indices in $\mathbf{C}_{i,k} = \sum_j \mathbf{A}_{i,j}\mathbf{B}_{j,k}$, the entries are placed at

$$\mathrm{pos}_{\mathbf{A}}(i,j) = i \cdot d_1 d_2 - j + n \cdot \mathbf{1}[i{=}0 \wedge j{>}0], \tag{8}$$

$$\mathrm{pos}_{\mathbf{B}}(j,k) = k \cdot d_1 + j, \tag{9}$$

with $\mathbf{A}_{0,j}$ negated for $j > 0$ so that the wrap-around's sign flip cancels out. These positions satisfy $\mathrm{pos}_{\mathbf{A}}(i,j) + \mathrm{pos}_{\mathbf{B}}(j,k) = i \cdot d_1 d_2 + k \cdot d_1 \pmod{n}$ *independently of* j, so the convolution accumulates the d_1 products $\mathbf{A}_{i,j}\mathbf{B}_{j,k}$ into the single coefficient $\mathrm{RLWE}(\mathbf{C})[\,i \cdot d_1 d_2 + k \cdot d_1\,] = \mathbf{C}_{i,k}$.

Toy example. Let $q = 2^7$, $d_0 = 4$, $d_1 = 3$, $d_2 = 2$, $n = 24$, so $d_0 d_1 d_2 = n$ exactly fills one ciphertext. For

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

$$\Longrightarrow \mathbf{A} \cdot \mathbf{B} \equiv \begin{pmatrix} 22 & 28 \\ 49 & 64 \\ 76 & 100 \\ 103 & 8 \end{pmatrix} \pmod{2^7}.$$

Equations (8) and (9) give the encodings (with arithmetic over $\mathbb{Z}_{2^7}$)

$$\mathrm{RLWE}(\mathbf{A}) = 1 + 6X^4 + 5X^5 + 4X^6 + 9X^{10} + 8X^{11} + 7X^{12}, \\ + 12X^{16} + 11X^{17} + 10X^{18} - 3X^{22} - 2X^{23}$$

$$\mathrm{RLWE}(\mathbf{B}) = 1 + 3X + 5X^2 + 2X^3 + 4X^4 + 6X^5.$$

($d_1 d_2 = 6$ between the first and second terms; $d_1 d_2 = 6$ between the second and third terms.)

Observe that $\mathbf{B}$ is laid out in column-major order. Moreover, $\mathbf{A}$ is neither row- nor column-major, but elements of each column are placed at a constant stride $d_1 d_2 = 6$ (modulo n). For example, following Equation (8), $\mathbf{A}_{0,0} = 1$ sits at X^0, and $\mathbf{A}_{1,0} = 4$ at X^6. Similarly in column 1, $\mathbf{A}_{0,1} = 2$ wraps past n and lands at X^{23} with a sign flip (as -2), and the next element of the column, $\mathbf{A}_{1,1} = 5$, lands at $X^{(23+6) \bmod 24} = X^5$.

Multiplying $\mathrm{RLWE}(\mathbf{A}) \cdot \mathrm{RLWE}(\mathbf{B})$ modulo $X^{n=24} + 1$ yields

$$\begin{aligned} \mathrm{RLWE}(\mathbf{C}) \equiv\ & \boxed{22} + 19X + 21X^2 + \boxed{28X^3} + 22X^4 + 29X^5 \\ & + \boxed{49X^6} + 49X^7 + 54X^8 + \boxed{64X^9} + 55X^{10} \\ & + 59X^{11} + \boxed{76X^{12}} + 79X^{13} + 87X^{14} + \boxed{100X^{15}} \\ & + 88X^{16} + 89X^{17} + \boxed{103X^{18}} + 109X^{19} + 120X^{20} \\ & + \boxed{8X^{21}} + 103X^{22} + 49X^{23}. \end{aligned}$$

To decode the result, observe that the boxed coefficients sit at the strided positions $i \cdot d_1 d_2 + k \cdot d_1 = 6i + 3k$, i.e. indices $\{0, 3, 6, 9, 12, 15, 18, 21\}$, and exactly recover the eight entries of $\mathbf{A} \cdot \mathbf{B}$ shown above. The remaining 16 coefficients of $\mathrm{RLWE}(\mathbf{C})$ (positions not divisible by $d_1 = 3$) are cross terms that get masked out after the operation.

Appendix G: Per-Layer rsqrt Mean

We used 40 GSM8K [21] prompts with 100 decode tokens to calibrate our per-layer constants. We then validated the end-to-end effect on Llama 3.1 8B [32, 63] and Gemma 2 9B [62] using the GSM8K and HumanEval [17] datasets, where a single wrong token can cascade through subsequent steps and corrupt the answer; we observed that both models' responses matched those of the original model exactly, *with no accuracy loss*. Notably, the constants calibrated on GSM8K (a math task) transferred unchanged to HumanEval (a coding task), indicating that the per-layer statistics are a property of the model rather than the workload.

Moreover, Figures 12 and 13 extend Figure 5 to all 32 layers of Llama 3.1 8B, showing $\frac{1}{\sqrt{s}}$ for every layer and prompt. The same pattern holds throughout: values cluster tightly around the per-layer mean precomputed from the calibration set.

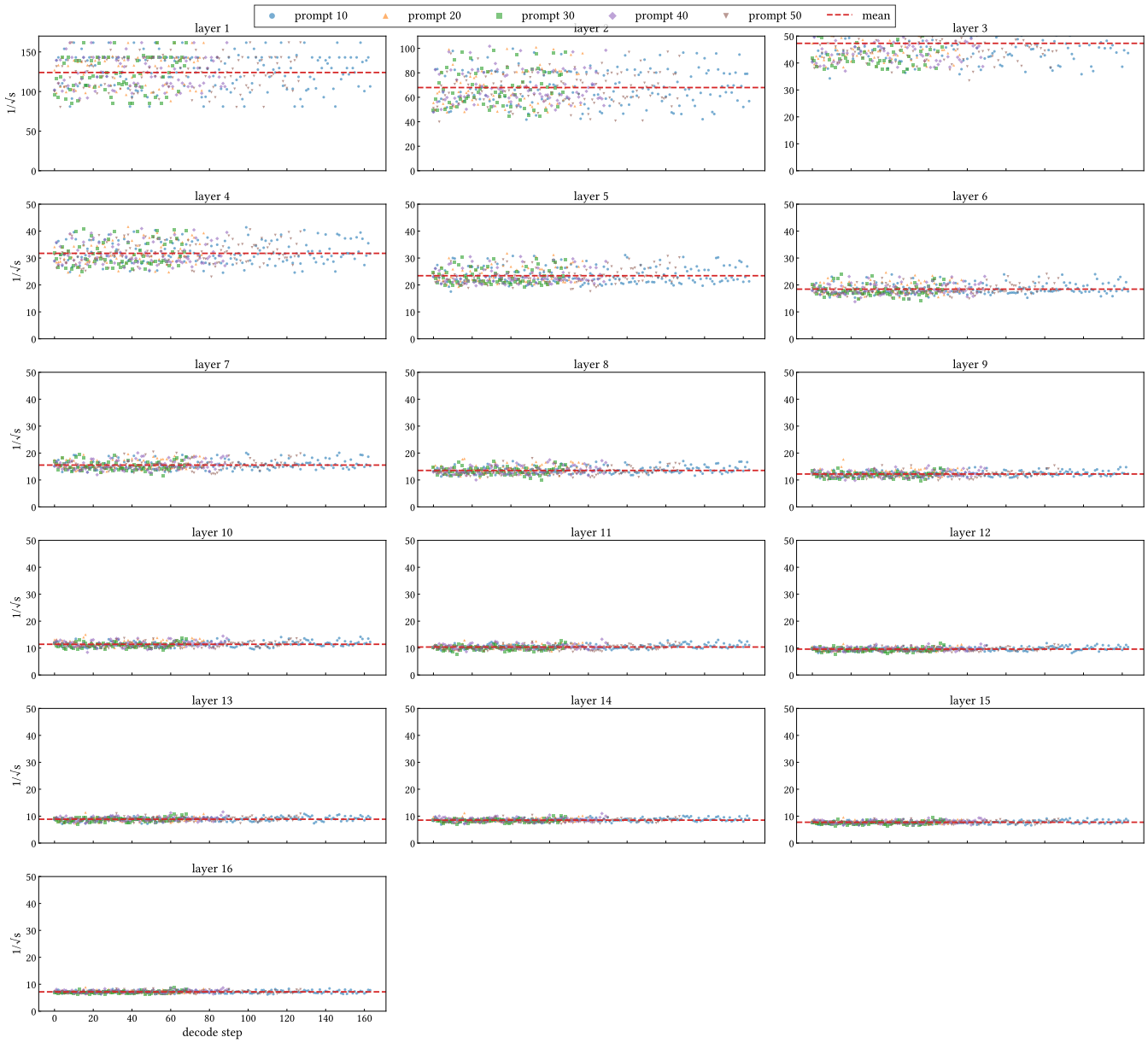


Figure 12: RMSNorm's $\frac{1}{\sqrt{s}}$ per prompt for layers 1–16 of Llama 3.1 8B.

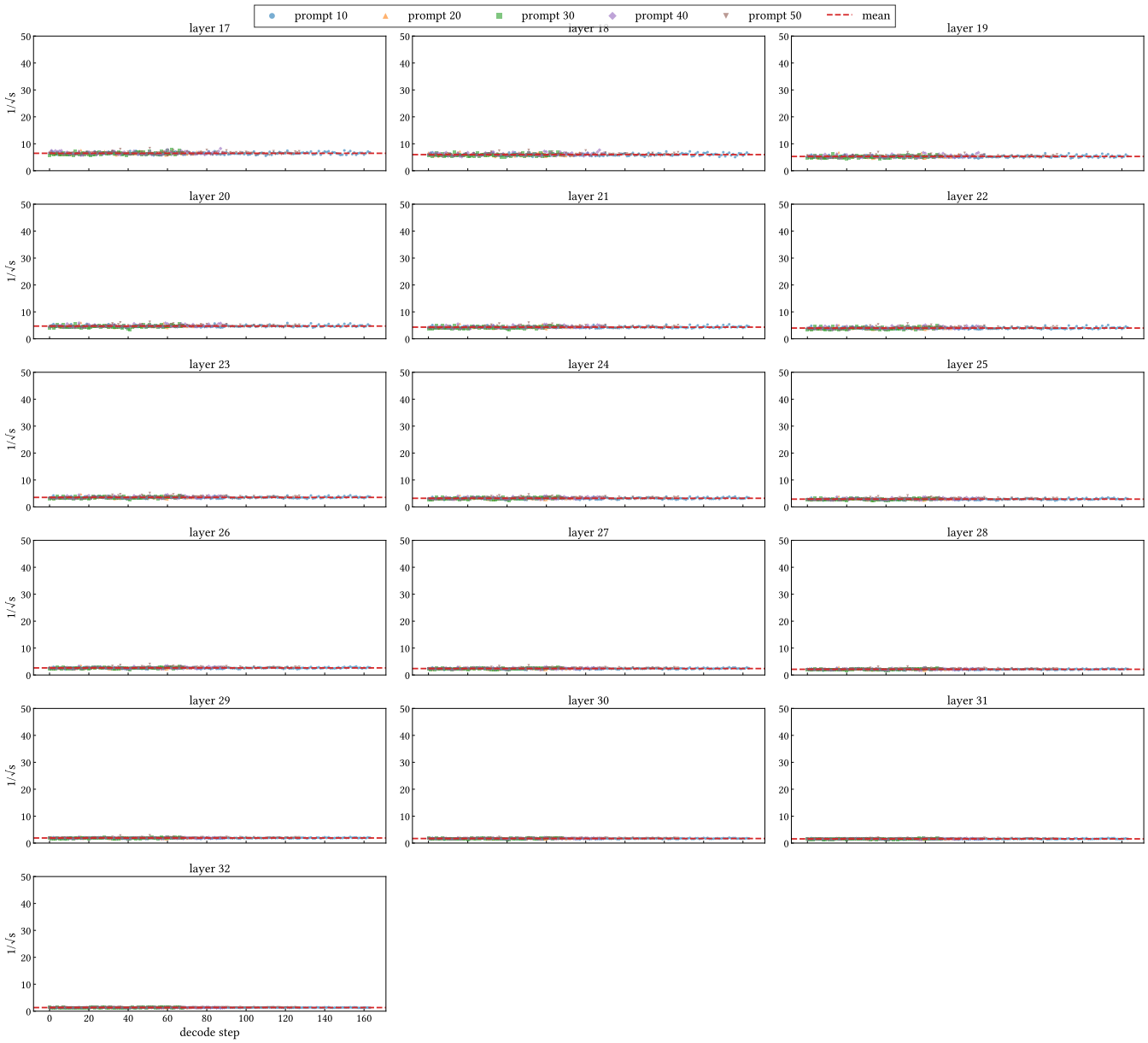


Figure 13: RMSNorm's $\frac{1}{\sqrt{s}}$ per prompt for layers 17–32 of Llama 3.1 8B.

Algorithm 1 CKKS Softmax with soft bootstrapping

Require: P_0, P_1 hold additive shares of max-subtracted logits $x_1, \ldots, x_{N_{\text{kv}}} \leq 0$; P_0 holds the CKKS secret key, P_1 evaluates homomorphically.

Ensure: Additive $\mathbb{Z}_p$ shares of $\text{softmax}(x)$.

Notation: $\text{Enc}(v)$ — *P_0 encrypts its share of v and sends it; P_1 adds its own share to obtain a fresh top-level ciphertext $[\![v]\!]$ of the reconstructed value.* $\text{SoftBootstrap}([\![v]\!])$ — *P_1 subtracts a fresh random mask from $[\![v]\!]$ and sends it; P_0 decrypts. The parties are left with additive integer shares of v.*

// Phase 1: exponential, $\exp(x) \approx (1 + x/2^{10})^{2^{10}}$*, two segments*

P_1: $[\![x]\!] \leftarrow \text{Enc}(x)$; $[\![u]\!] \leftarrow 1 + [\![x]\!]/2^{10}$; square $5\times$ ▷ 6 levels

$\text{SoftBootstrap}([\![u]\!])$

P_1: $[\![u]\!] \leftarrow \text{Enc}(u)$; square $5\times$ ▷ 5 levels

$\text{SoftBootstrap}([\![u]\!]) \rightarrow$ shares of $e_i = \exp(x_i)$

// Local: per-head sum (no communication)

Each party locally sums its shares: $\langle S \rangle \leftarrow \sum_i \langle e_i \rangle$

// Phase 2: reciprocal seed on the normalized sum $\tilde{s} = S/N_{\text{kv}}$; the exact max guarantees $\tilde{s} \in [1/N_{\text{kv}}, 1]$

P_1: $[\![\tilde{s}]\!] \leftarrow \text{Enc}(S)/N_{\text{kv}}$ ▷ 1 level

P_1: $[\![y]\!] \leftarrow p_5([\![\tilde{s}]\!])$ by Horner's rule, where p_5 interpolates $1/x$ at Chebyshev nodes on $[1/N_{\text{kv}}, 1]$ ▷ 5 levels

$\text{SoftBootstrap}([\![y]\!])$ ▷ $y \approx 1/\tilde{s} = N_{\text{kv}}/S$

// Phase 3: Newton refinement of $y \rightarrow 1/\tilde{s}$, two segments

P_1: $[\![y]\!] \leftarrow \text{Enc}(y)$; $[\![\tilde{s}]\!] \leftarrow \text{Enc}(S)/N_{\text{kv}}$ ▷ 1 level

P_1: $[\![y]\!] \leftarrow [\![y]\!]\,(2 - [\![\tilde{s}]\!]\,[\![y]\!])$, twice ▷ 2 levels each

$\text{SoftBootstrap}([\![y]\!])$

P_1: $[\![y]\!] \leftarrow \text{Enc}(y)$; $[\![\tilde{s}]\!] \leftarrow \text{Enc}(S)/N_{\text{kv}}$; one more Newton step ▷ $1+2$ levels

P_1: $[\![y]\!] \leftarrow [\![y]\!]/N_{\text{kv}}$ ▷ undo normalization: $y \approx 1/S$; 1 level

$\text{SoftBootstrap}([\![y]\!]) \rightarrow$ shares of S^{-1}

// Phase 4: final product, in the small $n = 2^{13}$ context

P_1: $[\![e]\!] \leftarrow \text{Enc}(e)$; $[\![S^{-1}]\!] \leftarrow \text{Enc}(S^{-1})$

P_1: $[\![\sigma]\!] \leftarrow [\![e]\!] \cdot [\![S^{-1}]\!]$ ▷ 1 level

$\text{SoftBootstrap}([\![\sigma]\!])$; both parties reduce their shares modulo $p \rightarrow$ additive $\mathbb{Z}_p$ shares of $\text{softmax}(x)$